

UNIT III

DATA ORGANIZATION

3.1 ELEMENTARY DATA ORGANIZATION

Data Structures:

Data may be organized in many different ways. The logical or mathematical model of a particular organization of data is called a data structure.

The choice of a particular data model depends on the two considerations:

1. It must be rich enough in structure to mirror the actual relationships of the data in the real world.
2. The structure should be simple enough that one can effectively process the data whenever necessary.

Data Structures are the main part of many computer science algorithms as they enable the programmers to handle the data in an efficient way. It plays a vital role in enhancing the performance of a software or a program as the main function of the software is to store and retrieve the user's data as fast as possible.

Basic Terminology: Elementary Data Organization:

- i. *Data*: Data are simply values or sets of values.
- ii. *Data items*: Data items refers to a single unit of values.

Data items that are divided into sub-items are called Group items. Ex: An Employee Name may be divided into three subitems- first name, middle name, and last name.

Data items that are not able to divide into sub-items are called Elementary items. Ex: SSN

- iii. *Entity*: An entity is something that has certain attributes or properties which may be assigned values. The values may be either numeric or non-numeric.

Ex: Attributes- Names, Age, Sex, SSN

Values- Rohland Gail, 34, F, 134-34-5533

Entities with similar attributes form an entity set. Each attribute of an entity set has a range of values, the set of all possible values that could be assigned to the particular attribute. The term “information” is sometimes used for data with given attributes, of, in other words meaningful or processed data.

- iv. *Field* is a single elementary unit of information representing an attribute of an entity.
- v. *Record* is the collection of field values of a given entity.
- vi. *File* is the collection of records of the entities in a given entity set.

Each record in a file may contain many field items but the value in a certain field may uniquely determine the record in the file. Such a field K is called a primary key and the values k1, k2,in such a field are called keys or key values. Records may also be classified according to length. A file can have fixed-length records or variable-length records. In fixed-length records, all the records contain the same data items with the same amount of space assigned to each data item. In variable-length records file records may contain different lengths. Example: Student records have variable lengths, since different students take different numbers of courses. Variable-length records have a minimum and a maximum length. The above organization of data into fields, records and files may not be complex enough to maintain and efficiently process certain collections of data. For this reason, data are also organized into more complex types of structures.

Need of Data Structure:

The structure of the data and the synthesis of the algorithm are relative to each other. Data presentation must be easy to understand so the developer, as well as the user, can make an efficient implementation of the operation.

Data structures provide an easy way of organising, retrieving, managing, and storing data. Here is a list of the needs for data.

- i. Data structure modification is easy.
- ii. It requires less time.
- iii. Save storage memory space.
- iv. Data representation is easy.
- v. Easy access to the large database

Data Type and Data Structure:

A data structure is not only used for organizing the data. It is also used for processing, retrieving, and storing data. Different basic and advanced types of data structures are used in almost every program or software system that has been developed. So we must have good knowledge of data structures.

Data structures are an integral part of computers used for the arrangement of data in memory. They are essential and responsible for organizing, processing, accessing, and storing data efficiently. But this is not all. Various types of data structures have their characteristics, features, applications, advantages, and disadvantages.

How Data Structure varies from Data Type:

We already have learned about data structure. Many times, what happens is that people get confused between data type and data structure. So let's see a few differences between data type and data structure to make it clear.

Data Type	Data Structure
The data type is the form of a variable to which a value can be assigned. It defines that the particular variable will assign the values of the given data type only.	Data structure is a collection of different kinds of data. That entire data can be represented using an object and can be used throughout the program.
It can hold value but not data. Therefore, it is data less.	It can hold multiple types of data within a single object.
The implementation of a data type is known as abstract implementation.	Data structure implementation is known as concrete implementation.
There is no time complexity in the case of data	In data structure objects, time complexity plays an important role.
In the case of data types, the value of data is not stored because it only represents the type of data that can be stored.	While in the case of data structures, the data and its value acquire the space in the computer's main memory. Also, a data structure can hold different kinds and types of data within one single object.
Data type examples are int, float, double, etc.	Data structure examples are stack, queue, tree, etc.

Categories of Data Structure:

Data structure has many different uses in our daily life. There are many different data structures that are used to solve different mathematical and logical problems. By using data structure, one can organize and process a very large amount of data in a relatively short period.

Linear data structure: Data structure in which data elements are arranged sequentially or linearly, where each element is attached to its previous and next adjacent elements, is called a linear data structure.

Examples of linear data structures are array, stack, queue, linked list, etc.

Static data structure: Static data structure has a fixed memory size. It is easier to access the elements in a static data structure.

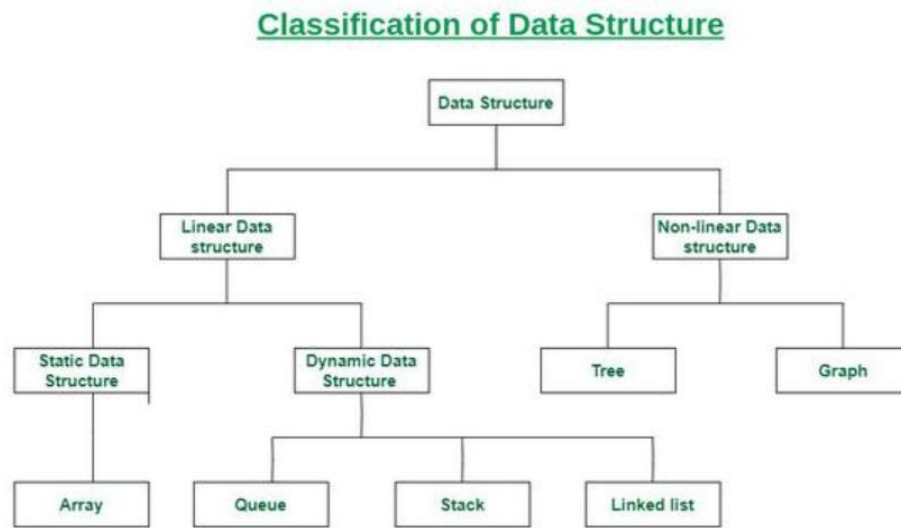
An example of this data structure is an array.

Dynamic data structure: In the dynamic data structure, the size is not fixed. It can be randomly updated during the runtime which may be considered efficient concerning the memory (space) complexity of the code.

Examples of this data structure are queue, stack, etc.

Non-linear data structure: Data structures where data elements are not placed sequentially or linearly are called non-linear data structures. In a non-linear data structure, we can't traverse all the elements in a single run only.

Examples of non-linear data structures are trees and graphs.



Classification of Data Structures

3.2 ABSTRACT DATA TYPE – ADT

Some data is associated with a set of operations and mathematical abstractions have been an integral part of programming languages, abstractions which we have been using without giving it much thought like integer, Boolean, etc. The basic idea of ADTs is a logical view of the data objects together with specifications of the operations required to create and manipulate them. Designing and coding an ADT once enables the reuse of the ADT. Any changes in the implementation are transparent to the other modules. Just like an algorithm is described using pseudo-code, a data structure can be described using ADT. The meaning of „abstract“ in Latin means to „pull out“ and in our context abstract essentially means to pull out the essentials and to defer or hide the details. In other words, abstraction emphasizes essentials and defers the details, making engineering artifacts easier to use. Figure 1.3 shows the difference between primitive data types and the abstract data types.

In other words, ADT can be described as follows:

1. Declaration of data
2. Declaration of operations
3. Encapsulation of data and operations

What is an Abstract Data Type (ADT)?

An Abstract Data Type (ADT) is a logical representation of data along with the operations that can be performed on the data. The key feature of an ADT is abstraction, which hides implementation details and focuses only on what the data does rather than how it does it.

Example: A Stack ADT provides operations like push (add an element), pop (remove an element), and peek (view the top element) without specifying how these operations are implemented (e.g., using arrays or linked lists).

Why Are ADTs Important?

- Abstraction: Simplifies complex systems by focusing on high-level functionality.
- Modularity: Facilitates the development of reusable and maintainable software.
- Separation of Concerns: Allows developers to change implementations without affecting the overall system.

ADT in Real Life

- Library System: Books (data) with operations such as borrow, return, and search.
- Bank Account: Balance (data) with operations such as deposit, withdraw, and transfer.

Components of ADTs

1. Data

- Defines the type of information to be stored (e.g., integers, strings, objects).
- Example: In a list, the data could be an array of integers, names, or more complex structures like objects.

2. Operations

- Specifies the set of operations that can be performed on the data. Operations include:
- Insertions and deletions
- Searching and sorting, & Traversing and retrieving elements

3. Interface vs. Implementation

- Interface: The user-facing contract that specifies what operations can be performed.
- Example: A queue ADT provides enqueue and dequeue operations.
- Implementation: The internal mechanism, which can vary based on requirements.
- Example: A queue can be implemented using arrays or linked lists.

Common Abstract Data Types:

1. List ADT

A collection of ordered elements where duplicates are allowed.

Operations:

- Insert: Add an element at a specific position.
- Delete: Remove an element by position or value.
- Traverse: Access elements in order.

Implementations:

- Array-based List: Uses a contiguous memory block. Efficient for random access.
- Linked List: Nodes with pointers. Suitable for dynamic memory requirements.

Applications:

- To-do lists
- Inventory systems
- Playlist management

2. Stack ADT

A LIFO (Last In, First Out) data structure where the last inserted element is removed first.

Operations:

- Push: Add an element to the top of the stack.
- Pop: Remove the top element.
- Peek: View the top element without removing it.

Implementations:

- Array-based Stack: Fixed size, fast access.
- Linked List Stack: Dynamic size, slower but flexible.

Applications:

- Expression evaluation
- Function call stack in programming
- Undo/redo functionality

3. Queue ADT

A FIFO (First In, First Out) data structure where the first inserted element is removed first.

Operations:

- Enqueue: Add an element to the rear of the queue.
- Dequeue: Remove an element from the front of the queue.

Variants:

- Circular Queue: Avoids wasted space by reusing memory.

- Priority Queue: Processes elements based on priority rather than order.
- Deque: Allows insertion and deletion at both ends.

Applications:

- Job scheduling in operating systems
- Order processing systems
- Customer service lines

4. Set ADT

A collection of unique elements.

Operations:

- Union: Combine all elements from two sets.
- Intersection: Extract common elements.
- Difference: Extract elements unique to one set.

Applications:

- Membership checking
- Data deduplication
- Cryptographic systems

5. Map (Dictionary) ADT

- A collection of key-value pairs.

Operations:

- Insert(key, value): Add a key-value pair.
- Delete(key): Remove a key-value pair.
- Search(key): Retrieve the value associated with a key.

Applications:

- Implementing databases
- Caching systems
- Indexing in search engines

Abstract Data Models:

An Abstract Data Model describes how ADTs can be used to design data-centric systems. It separates logical functionality from implementation details, enabling developers to focus on problem-solving rather than low-level coding.

Benefits of Abstract Data Models

- Flexibility: Easy to adapt to different use cases by switching implementations.
- Scalability: Supports the growth of systems without significant rework.
- Code Reusability: Promotes the use of predefined ADTs in various applications.

Implementation of ADTs:

ADTs can be implemented in various ways, depending on performance requirements. Some examples include:

- Array Implementation: Fixed size, fast access, and simple structure.
- Linked List Implementation: Dynamic size, suitable for scenarios with frequent insertions and deletions.
- Tree Implementation: Hierarchical organization, efficient for searches.

Applications of ADTs

- Software Development: Modular design and reusable components.
- Database Systems: Efficient data retrieval and management.
- Network Routing: Priority queues for optimal path calculation.

Abstract Data Types and Models form the foundation of data structure design and software development. By focusing on abstraction and modularity, ADTs empower developers to build efficient, scalable, and maintainable systems. They serve as the bridge between theoretical computer science and practical software engineering.

3.3 FUNDAMENTALS OF LINEAR DATA STRUCTURES

Linear Data Structure:

It is a type of data structure where the arrangement of the data follows a linear trend. The data elements are arranged linearly such that the element is directly linked to its previous and the next elements. As the elements are stored linearly, the structure supports single-level storage of data. And hence, traversal of the data is achieved through a single run only.

Characteristics:

- It is a type of data structure where data is stored and managed in a linear sequence.
- Data elements in the sequence are linked to one after the other.
- Implementation of the linear structure of data in a computer's memory is easy as the data is organized sequentially.
- Array, queue, Stack, linked list, etc. are examples of this type of structure.
- The data elements stored in the data structure have only one relationship.

- Traversal of the data elements can be carried out in a single run as the data elements are stored in a single level.
- There is poor utilization of the computer memory if a structure storing data linearly is implemented.
- With the increase in the size of the data structure, the time complexity of the structure increases.

These structures can therefore be summarized as a type of data structure where the elements are stored sequentially and follow the order where:

- Only one first element is present which has one next element.
- Only one last element is present which has one previous element.
- All the other elements in the data structure have a previous and a next element

You can use linear data structure in C, C++, Python, JavaScript, or any other programming language you are familiar with.

If you get baffled when given a list of linear structures and wonder which of the following is a linear data structure, here's a rundown of the types.

Types of Linear Data Structures:

Operations performed on linear data structure include insertion, deletion, searching, traversing, and sorting. All these operations serve as the foundation for linear data structures.

Array:

The array is that type of structure that stores homogeneous elements at memory locations which are contiguous. The same types of objects are stored sequentially in an array. The main idea of an array is that multiple data of the same type can be stored together. Before storing the data in an array, the size of the array has to be defined. Any element in the array can be accessed or modified and the elements stored are indexed to identify their locations.

An array can be explained with the help of a simple example of storing the marks for all the students in a class. Suppose there are 20 students, then the size of the array has to be mentioned as 20. Marks of all the students can then be stored in the created array without the need for creating separate variables for marks for every student. Simple traversal of the array can lead to the access of the elements.

Arrays incorporate the use of zero-based indexing techniques. This means users can access the first element with an index of 0, the second with an index of 1, and so on. Another remarkable feature of this linear data structure is that arrays provide a constant time of $O(1)$ to access the elements, which means that it takes equal time to access any element in the series, disregarding the size of the array.

Types of Arrays:

- One-dimensional array: This is a simple form of array that contains elements, all of which are the same type of data, in a single row.
- Two-dimensional array: This is also known as a matrix. This type of data structure has rows and columns and appears like a grid. The elements can be accessed using two indices- one for column and one for row.
- Multi-dimensional array: These arrays have more than two dimensions.

Operations Performed on Arrays:

The following operations can be performed on arrays:

- Accessing an element: Accessing an element by its index is an essential operation that can be performed on arrays. It is a constant time operation with a time complexity of $O(1)$.
- Inserting or deleting elements: Inserting elements at the end of an array is a constant-time operation having complexity $O(1)$. But inserting an element at the beginning takes $O(n)$ time since all the elements have to be shifted.
- The same goes for deletion of elements in an array.
- Searching for elements: For unsorted data, linear search takes $O(n)$ time, and for sorted data, binary search takes $O(\log n)$ time.

Linked list:

The linked list is that type of data structure where separate objects are stored sequentially. Every object stored in the data structure will have the data and a reference to the next object. The last node of the linked list has a reference to null. The first element of the linked list is known as the head of the list. There are many differences between a linked list to the other types of data structures. These are in terms of memory allocation, the internal structure of the data structure, and the operations carried on the linked list.

Getting to an element in a linked list is a slower process compared to the arrays as the indexing in an array helps in locating the element. However, in the case of a linked list, the process has to start from the head and traverse through the whole structure until the desired element is reached. In contrast to this, the advantage of using linked lists is that the addition or deletion of elements at the beginning can be done very quickly.

There are three types of linked lists:

1. Single Linked List: This type of structure has the address or the reference of the next node stored in the current node. Therefore, a node which at the last has the address and reference as a NULL. Example: A->B->C->D->E->NULL.
2. A Double Linked List: As the name suggests, each node has two references associated with it. One reference directs to the previous node while the second reference points to

the next node. Traversal is possible in both directions as reference is available for the previous nodes. Also, explicit access is not required for deletion. Example: $\text{NULL} \leftarrow \text{A} \leftrightarrow \text{B} \leftrightarrow \text{C} \leftrightarrow \text{D} \leftrightarrow \text{E} \rightarrow \text{NULL}$.

3. **Linked List which is circular:** The nodes in a circular linked list are connected in a way that a circle is formed. As the linked list is circular there is no end and hence no NULL. This type of linked list can follow the structure of both singly or doubly. There is no specific starting node and any node from the data can be the starting node. The reference of the last node points towards the first node. Example: $\text{A} \rightarrow \text{B} \rightarrow \text{C} \rightarrow \text{D} \rightarrow \text{E}$.

Properties of a linked list are:

- Access time: $O(n)$
- Searching time: $O(n)$
- Adding element: $O(1)$
- Deleting an Element : $O(1)$

Stack:

The stack is another type of structure where the elements stored in the data structure follow the rule of LIFO (last in, first out) or FILO (First In Last Out). Two types of operations are associated with a stack i.e. push and pop. Push is used when an element has to be added to the collection and pop is used when the last element has to be removed from the collection. Extraction can be carried out for only the last added element.

Types of stack:

There are two types of stacks:

- **Fixed-size stack:** This kind of stack does not grow or shrink. Once full, any attempt to add an element will lead to an overflow error. Similarly, an attempt to remove an element will also display an underflow error.
- **Dynamic size stack:** This kind of stack can grow or shrink. When a stack is full or empty, it can automatically resize to accommodate a new element or shrink in size.

Other operations are:

- **top():** This operation helps to return the element that has been inserted last and is at the top without removing it.
- **size():** This operation indicates the total number of elements the stack contains.
- **isEmpty():** This operation helps to identify if a stack is empty.

Properties of a stack are:

- Adding element: $O(1)$
- deleting element: $O(1)$

- Accessing Time: $O(n)$ [Worst Case]

Only one end allows inserting and deleting an element.

Queue:

Queue is the type of data structure where the elements to be stored follow the rule of First In First Out (FIFO). The particular order is followed for performing the required operations over the elements. The difference of a queue from that of a stack lies in the removal of an element, where the most recently added object is removed first in a stack. Whereas, in the case of a queue, the element that was added first is removed first.

Following is a list of the different types of queues:

- Input restricted queue: In this kind of queue, one can only insert inputs from one end. Deletion, however, can be done from both ends.
- Output restricted queue: This is just the reverse of input restricted queues. Here, the input can be taken from both ends, but deletion can only be done from one end.
- Circular queue: In this kind of queue, the first and the last positions are connected to one another, resulting in a circular structure.
- Double-ended queue: This kind of operation supports insertion and deletion from both ends.
- Priority queue: In this kind of queue, elements can be accessed based on priority assigned to them.

Both the end of the data structure is used for the insertion and the removal of data. The two main operations governing the structure of the queue are enqueue, and dequeue. Enqueue refers to the process where inserting an element is allowed to the collection of data and dequeue refers to the process where removal of elements is allowed, which is the first element in the queue in this case.

Properties of a queue are:

- Inserting an element: $O(1)$
- Deleting an element: $O(1)$
- Accessing Time: $O(n)$

Other queue operations are:

- peek() or front(): This helps to acquire the data element available at the queue's front node without actually eliminating it.
- rear(): This operation returns an element at the rear without it being removed.
- isEmpty(): Finds out if a queue is empty.
- isFull(): Finds out if a queue is full.

Example of the queue: Similar to those queues made while waiting for the bus or anywhere, the data structure too follows the same pattern. We can imagine a person waiting for the bus and standing at the first position as the person that came to the queue first. This person will be the first one who will get onto a bus, i.e. exit the queue. Queues are applied when multiple users are sharing the same resources and they have to be served on the basis of who has come first on the server.

String:

Although non-modifiable, Strings display characteristics similar to those that linear data structures possess. They allow operations such as searching, concatenation, and substring extraction. Strings are used for input/output operations, data manipulation, and text processing.

Pros and Cons of Linear Data Structures

Some advantages and disadvantages associated with linear data structures make them suitable or unsuitable for the scenarios in which they operate. Understanding these advantages and disadvantages helps to choose the most appropriate linear data structure.

Advantages of linear data structures are as follows:

- Linear data structures help to access the elements sequentially.

For example, arrays facilitate efficient transversal with the help of loops. This makes it suitable for tasks requiring data processing in a linear pattern.

- Data can be inserted or deleted efficiently.

Arrays make it easy to access elements by index, and linked lists help insert or delete data at the beginning or end of a given list.

- Linear data structures are easy to understand and implement.

For instance, arrays help to store elements in a straightforward way, and linked lists use pointers that help to connect the nodes.

- Linear data structures offer flexibility as they allow dynamic resizing.

This facilitates easy accommodation of the varying data amounts. It especially holds true for linked lists, which can easily grow or shrink in size by removing or adding the nodes as and when required.

Disadvantages associated with linear data structures:

During the initialization of arrays, they have a fixed size determined.

As a result, the flexibility is limited. If the size is too small to store any additional elements, the entire array may be resized.

- There are insufficient search operations in linear data structures.

For instance, in the case of linked lists and arrays, one has to traverse the elements one by one until the desired result is obtained. The time complexity of $O(n)$ in the case of linear search may lead to inefficient results if the data set is large.

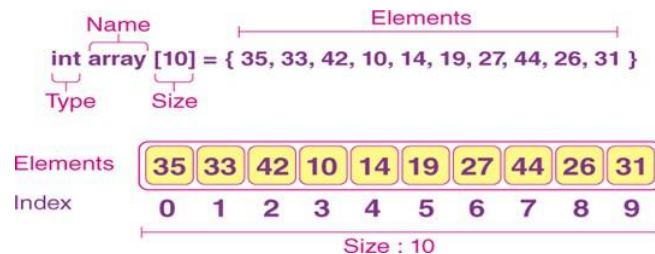
- Linked lists need more memory to store pointers than arrays.

This increases the overall memory usage of the data structure and causes the efficiency to decrease.

3.3.1 ARRAYS AND ARRAY OPERATIONS

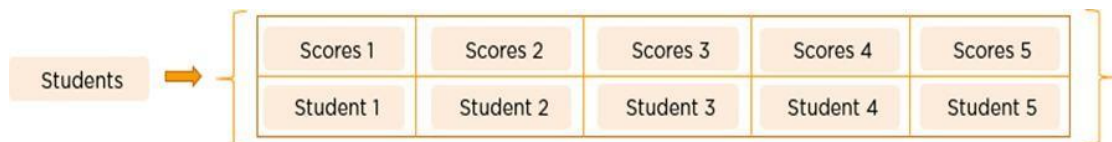
Arrays in Data Structure:

Arrays in data structures help solve some high-level problems like the "longest consecutive subsequence" program or some easy tasks like arranging the same things in ascending order. The concept is to collect many objects of the same kind.



An array is a linear data structure that collects elements of the same data type and stores them in contiguous and adjacent memory locations. Arrays work on an index system starting from 0 to $(n-1)$, where n is the size of the array. It is an array, but there is a reason that arrays came into the picture.

Why Do You Need an Array in Data Structures?



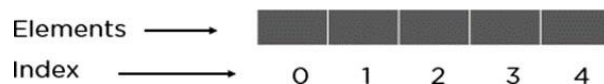
Let's suppose a class consists of ten students, and the class has to publish their results. If you had declared all ten variables individually, it would be challenging to manipulate and maintain the data.

If more students were to join, it would become more difficult to declare all the variables and keep track of it. To overcome this problem, arrays came into the picture.

What Are the Types of Arrays?

There are majorly two types of arrays, they are:

One-Dimensional Arrays:



You can imagine a 1d array as a row, where elements are stored one after another.

Multi-Dimensional Arrays:

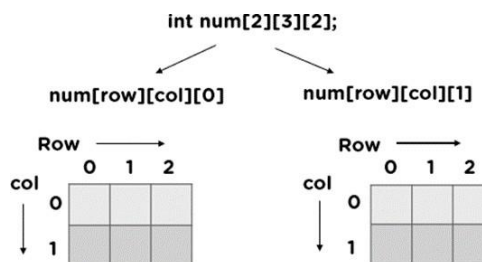
These multi-dimensional arrays are again of two types. They are:

Two-Dimensional Arrays:

Col →		0	1	2
Row ↓	0	1	2	3
	1	4	5	6
	2	7	8	9

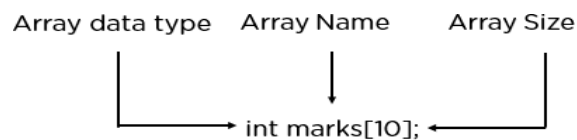
You can imagine it like a table where each cell contains elements.

Three-Dimensional Arrays:



You can imagine it like a cuboid made up of smaller cuboids where each cuboid can contain an element.

How Do You Declare an Array?



Arrays are typically defined with square brackets with the size of the arrays as its argument.

Here is the syntax for arrays:

1D Arrays: `int arr[n];`

2D Arrays: `int arr[m][n];`

3D Arrays: `int arr[m][n][o];`

How Do You Initialize an Array?

You can initialize an array in four different ways:

Method 1:

`int a[6] = {2, 3, 5, 7, 11, 13};`

Method 2:

```
int arr[] = {2, 3, 5, 7, 11};
```

Method 3:

```
int n; scanf("%d",&n); int arr[n];
```

```
for(int i=0;i<5;i++)
```

```
{
```

```
scanf("%d",&arr[i]); }
```

Method 4:

```
int arr[5]; arr[0]=1;
```

```
arr[1]=2;
```

```
arr[2]=3;
```

```
arr[3]=4;
```

```
arr[4]=5;
```

How Can You Access Elements of Arrays in Data Structures?

5	10	25	30	50
0	1	2	3	4

You can access elements with the help of the index at which you stored them. Let's discuss it with a code:

```
#include<stdio.h> int main()
```

```
{
```

```
int a[5] = {2, 3, 5, 7, 11};
```

```
printf("%d\n",a[0]); // we are accessing printf("%d\n",a[1]);
```

```
printf("%d\n",a[2]);
```

```
printf("%d\n",a[3]);
```

```
printf("%d",a[4]);
```

```
return 0;
```

```
}
```



```

2
3
5
7
11
-----
Process exited after 0.1476 seconds with return value 0
Press any key to continue . . .

```

In this “Arrays in Data structures” tutorial, you have seen the basics of array implementation, now you will perform operations on arrays.

Array Operations

- Traversal
- Insertion
- Deletion
- Searching
- Sorting

Traversing the Array:



Traversal in an array is a process of visiting each element once.

Code:

```

#include<stdio.h> int main()
{
int a[5] = {2, 3, 5, 7, 11};
for(int i=0;i<5;i++)
{
//traversing ith element in the array printf(“%d\n”,a[i]);
}
return 0; }

```

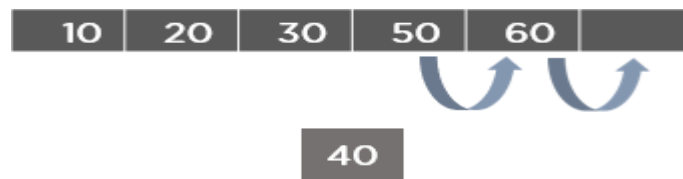
```

2
3
5
7
11
-----
Process exited after 0.1476 seconds with return value 0
Press any key to continue . . .

```

Insertion:

Insertion in an array is the process of including one or more elements in an array.



Insertion of an element can be done:

- At the beginning
- At the end and
- At any given index of an array.

At the Beginning: Code: #include<stdio.h>

```

int main()
{
    int array[10], n,i, item; printf("Enter the size of array: "); scanf("%d", &n);
    printf("\nEnter Elements in array: "); for(i=0;i<n;i++)
    {
        scanf("%d", &array[i]);
    }
    printf("\n enter the element at the beginning"); scanf("%d", &item);
    n++;
    for(i=n; i>1; i--)
    {
        array[i-1]=array[i-2];
    }
}

```

```

array[0]=item;
printf("resultant array element"); for(i=0;i<n;i++)
{
printf("\n%d", array[i]);
}
getch(); return 0; }

```

```

Enter the size of array: 5
Enter Elements in array: 2 3 5 7 11
enter the element at the beginning:1
resultant array element:
1
2
3
5
7
11
-----
Process exited after 19.94 seconds with return value 0
Press any key to continue . . .

```

At the End:

```

Code: #include<stdio.h> #include<conio.h> int main()
{
int array[10], i, values; printf("Enter 5 Array Elements: "); for(i=0; i<5; i++)
    scanf("%d", &array[i]); printf("\nEnter Element to Insert: "); scanf("%d", &values);
array[i] = values;
printf("\nThe New Array is:\n"); for(i=0; i<6; i++)
    printf("%d ", array[i]); getch();
return 0;
}

```

```

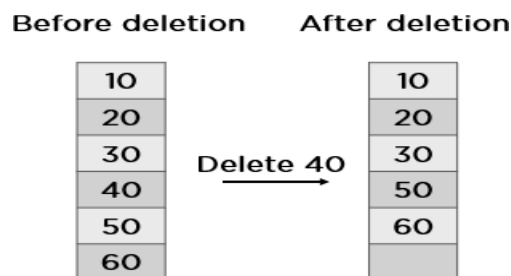
Enter 5 Array Elements: 2 3 5 7 11
Enter Element to Insert: 13
The New Array is:
2 3 5 7 11 13
-----
Process exited after 16.76 seconds with return value 0
Press any key to continue . . .

```

At a Specific Position:**Code:**

```
#include <stdio.h> int main()
{
    int array[100], pos, size, val; printf("Enter size of the array:"); scanf("%d", &size);
    printf("\nEnter %d elements\n", size); for (int i = 0; i < size; i++) scanf("%d", &array[i]);
    printf("Enter the insertion location\n"); scanf("%d", &pos);
    printf("Enter the value to insert\n"); scanf("%d", &val);
    for (int i = size - 1; i >= pos - 1; i--) array[i+1] = array[i];
    array[pos-1] = val; printf("Resultant array is\n"); for (int i = 0; i <= size; i++) printf("%d\n",
    array[i]); return 0;
}
```

```
Enter size of the array:5
Enter 5 elements
2 3 5 7 11
Enter the insertion location
2
Enter the value to insert
13
Resultant array is
2
13
3
5
7
11
-----
Process exited after 14.82 seconds with return value 0
Press any key to continue . . .
```

Deletion:

Deletion of an element is the process of removing the desired element and re-organizing it. You can also do deletion in different ways:

- At the beginning
- At the end

At the Beginning:

```
#include<stdio.h> int main()
{
int n,array[10];

printf("enter the size of an array"); scanf("%d" ,&n);
printf("enter elements in an array"); for(int i=0;i<n;i++)

    scanf("%d", &array[i]); n--;

for(int i=0;i<n;i++) array[i]=array[i+1]; printf("\nafter deletion "); for(int i=0;i<n;i++)
printf("\n%d" , array[i]);
}
```

```
enter the size of an array: 5
enter elements in an array: 2 3 5 7 11

after deletion:
3
5
7
11

-----
Process exited after 7.997 seconds with return value 0
Press any key to continue . . .
```

At the End: #include<stdio.h> int main()

```
{
int n,array[10];

printf("enter the size of an array"); scanf("%d" ,&n);
printf("enter elements in an array"); for(int i=0;i<n;i++)

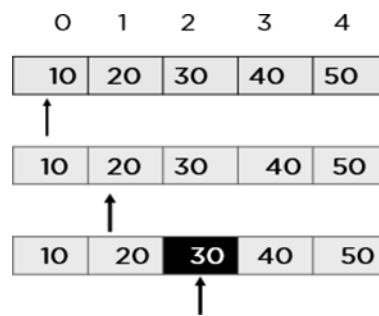
scanf("%d", &array[i]);

printf("\nafter deletion array elements are"); for(int i=0;i<n-1;i++)
printf("\n%d" , array[i]);
}
```

```
enter the size of an array:5
enter elements in an array:2 3 5 7 11

after deletion array elements:
2
3
5
7

-----
Process exited after 15.99 seconds with return value 0
Press any key to continue . . .
```

Searching for an Element

The method of searching for a specific value in an array is known as searching. There are two ways we can search in an array, they are:

- Linear search
- Binary search

Linear Search:**Code:**

```
#include <stdio.h>

int linear(int a[], int n, int x)
{
    for (int i = 0; i < n; i++) if (a[i] == x)
        return i; return -1;
}

int main(void)
{
    int a[] = { 2, 3, 4, 10, 40 };
    int x = 10;
    int n = sizeof(a) / sizeof(a[0]);

    // Function call

    int result = linear(a, n, x); if(result == -1)
    {
        printf("Element is not present in array");
    }
    else
```

```

{
printf("Element found at index: %d", result);
}
return 0;
}

```

```

Element found at index: 3
-----
Process exited after 0.108 seconds with return value 0
Press any key to continue . . .

```

Binary Search:

```

#include <stdio.h>

int binary(int a[], int lt, int rt, int k)
{
if (rt >= lt) {
int mid = lt + (rt - 1) / 2;
// check If element is at the middle if (a[mid] == k)
return mid;
//check if element is at the left side of mid if (a[mid] > x)
return binary(a, lt, mid - 1, k);
// Else element is at the right side of mid return binary(a, mid + 1, rt, k);
}
// if element not found return -1;
}

int main(void)
{
int a[] = { 2, 3, 5, 7, 11 };
int n = sizeof(a) / sizeof(a[0]); int k = 11;
int res = binary(arr, 0, n - 1, k); if(res == -1)
{
printf("Element is not found")
}
}

```

```

else
{
printf("Element found at index %d",res);
}
return 0;
}

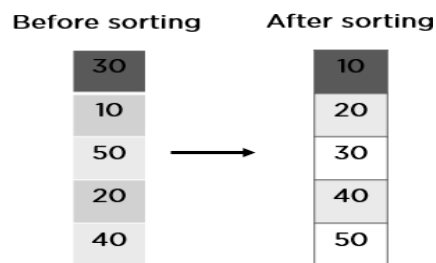
```

```

Element found at index 4
-----
Process exited after 0.08186 seconds with return value 0
Press any key to continue . . .

```

Sorting:



Sorting in an array is the process in which it sorts elements in a user-defined order.

Code:

```

#include <stdio.h> void main()
{
int temp, size, array[100]; printf("Enter the size \n"); scanf("%d", &size); printf("Enter the
numbers \n"); for (int i = 0; i < size; ++i) scanf("%d", &array[i]);
for (int i = 0; i < size; ++i)
{
for (int j = i + 1; j < size; ++j)
{
if (array[i] > array[j])
{
temp = array[i]; array[i] = array[j]; array[j] = temp;
} } }
printf("sorted array:\n"); for (int i = 0; i < size; ++i) printf("%d\n", array[i]);
}

```

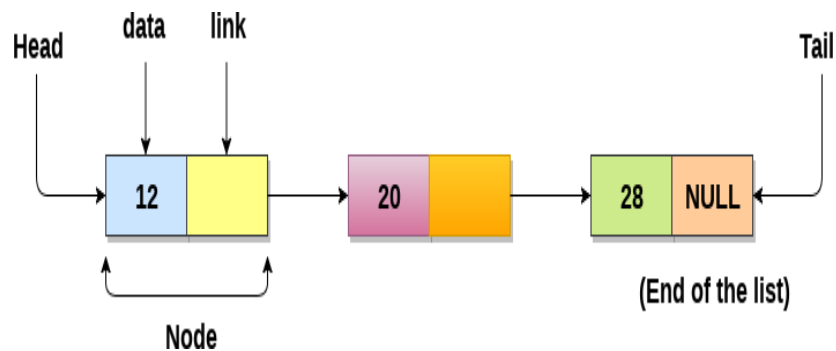

3.3.2 LIST AND OPERATIONS IN LINKED LIST

Linked List:

A linked list is a type of data structure in which elements are connected to each other through links or pointers. Each element (called a node) has two parts - the data and a link pointing to the next node.

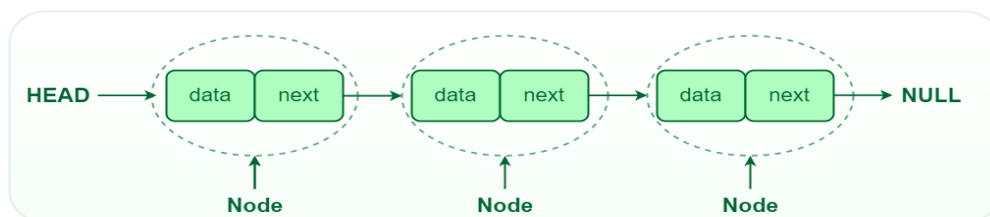
Definition: A linked list is a type of linear data structure that occurs when the data elements do not reside in consecutive memory regions. A linked list's components are connected using pointers (entities that lead to the next member).

A linked list comprises of nodes, each of which has an information field and a reference (link) to the subsequent node in that list.



Need of Linked Lists:

A linked list is a linear data structure that dynamically stores a group of data segments. These data segments are represented by nodes, which are connected by links or pointers. Each node has information stored in a linked list and a reference to the location of its subsequent node. The final node has null in its second field to avoid it will be pointing to no node.

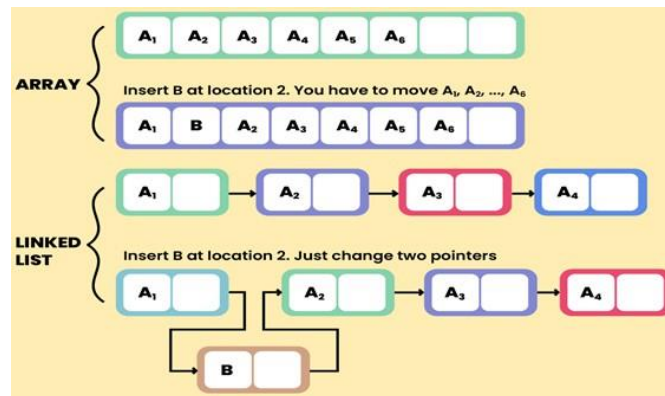


A linked list's size can be increased or decreased as needed. It does not take up much memory space. The question is fairly simple - If we have a linear data structure in arrays, why do we even need linked lists?

Because, in an array, it's much more difficult to insert or add an element. Even in a dynamic array, all the other elements would have to shift in order to make space if we have to add an element at the start of the index.

Also, there is a possibility that you'd need to allocate more memory space to the array before inserting an element.

This is where linked lists find their significance-



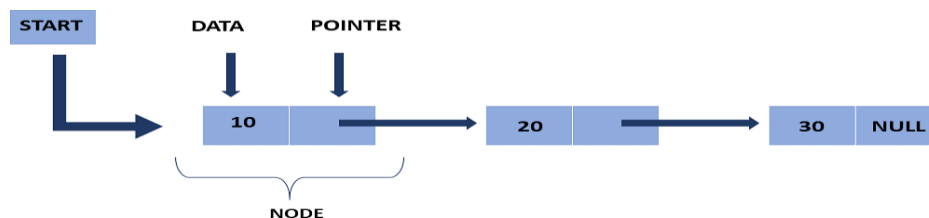
Representation of linked lists:

We can visualize a linked list as a chain of nodes; where every node contains a data field and a reference link to the next node in the list.

The first node in the sequence is termed as 'head'. And we can identify the last node as the one which points to 'Null'.

Each node consists of -

- A data field
- Reference to the next node



Structure of a node in C language-

```
struct node
{
    int data;
    struct node *next;
};
```

Let's make a simple linked list with three items to understand this better.

```
/* Initialize nodes */ struct node *head; struct node *one = NULL; struct node *two = NULL;
struct node *three = NULL;
/* Allocate memory */
```

```

one = malloc(sizeof(struct node)); two = malloc(sizeof(struct node)); three =
malloc(sizeof(struct node));

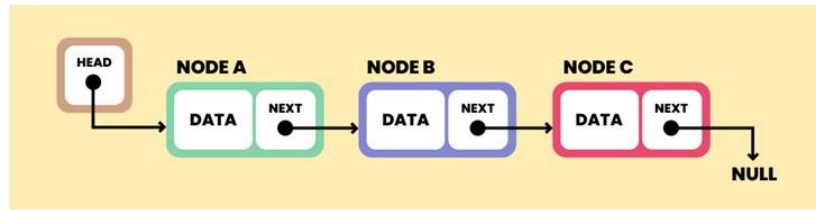
/* Assign data values */ one->data = 1;

two->data = 2; three->data=3;

/* Connect nodes */ one->next = two; two->next = three; three->next = NULL;

/* Save address of first node in head */ head = one;

```



Operations in a linked list:

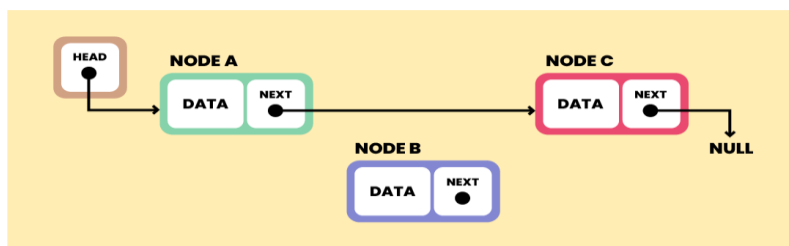
Following are the various operations to perform the required action on a linked list:

- Traversal - To access each element of the linked list.
- Insertion - To add/insert a new node to the list.
- Deletion - To remove an existing node from the list.
- Search - To find a node in the list.
- Sort - To sort the nodes.

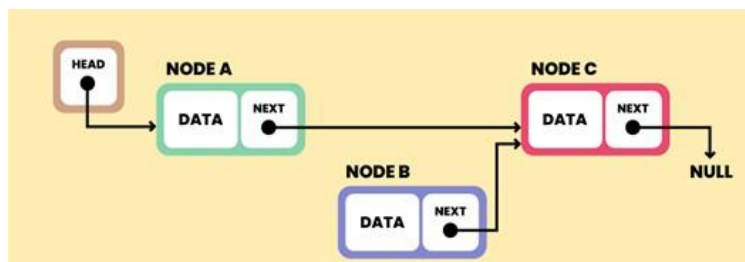
Now, let's look at each of these operations separately-

Insertion after specified node:

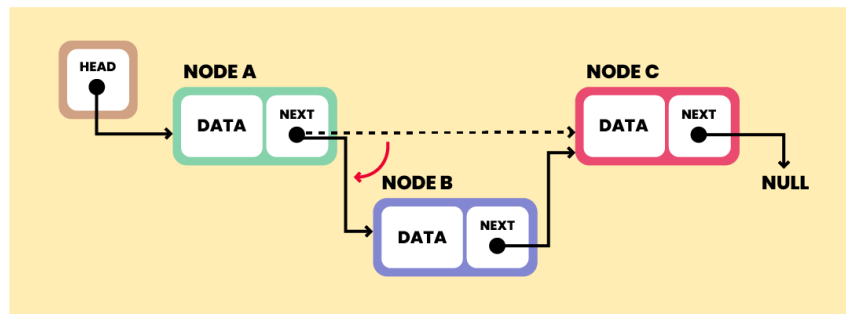
It's more than a one-step activity. First, we create a node with the same structure and specify the location where we'll insert it.



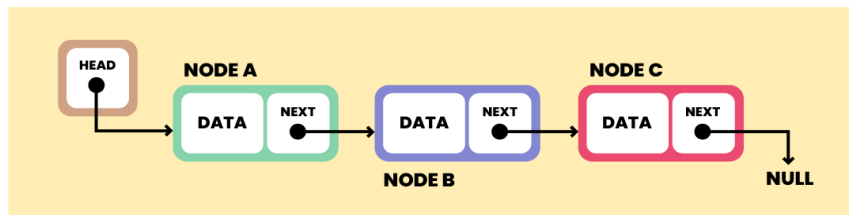
Suppose, we have to insert Node B between the two nodes A and C.



The new Node B will point to Node C. NodeB.next - Node C



Now, Node A should point to Node B NodeA.next — NodeB

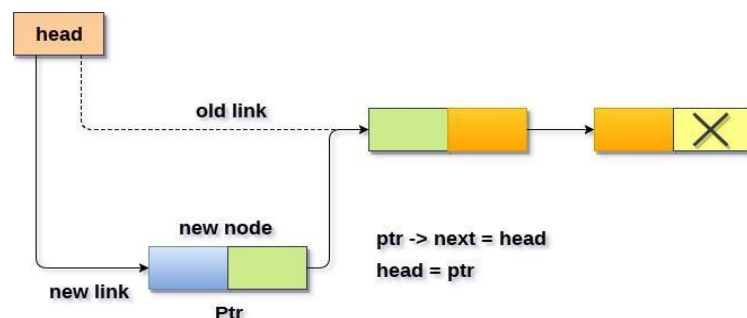


Doing this will place the new node in the middle of the two nodes; as we wanted. If we have to insert a node at the beginning of the list; the new node should point to the Head (First). Similarly, if we have to insert a node at the end; the last node should be made to point to the new node, and the new node should be pointing to 'Null'.

Insertion at the Beginning of the Linked List

Steps to Insert a node at the Beginning of the Linked List are,

- Create a new node.
- Initialize the data of this new node.
- Update the next variable of the new node as head.
- Update the head as the new node.



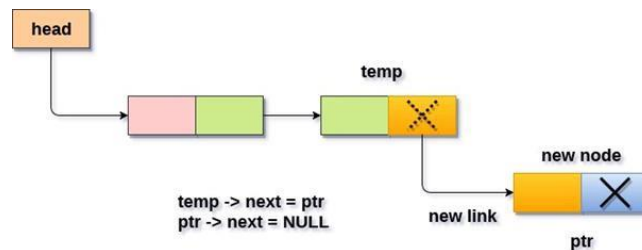
Insertion in singly linked list at the end

Steps to insert a node at the End of the Linked List are,

- Create a new node and initialize the data of this new node.
- Create another node, temp with the value head.

- Iterate over the Linked List while temp's next variable is not equal to NULL.
- Now temp is at the tail of the Linked List and we have to just update the next variable of temp to the new node.

Also make sure to update the next variable of the new node as NULL.



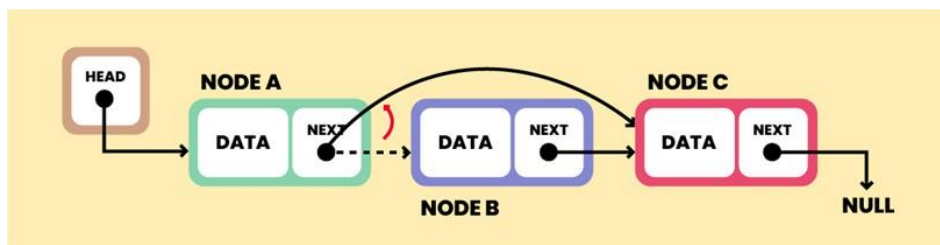
Inserting node at the last into a non-empty list

Deletion

Just like insertion, deletion is also a multi-step process. First, we track the node to be removed, by using searching algorithms.

Once, it's located, we'll change the reference link of the previous node to the node that is next to our target node.

In this case, Node A will now point to Node C.

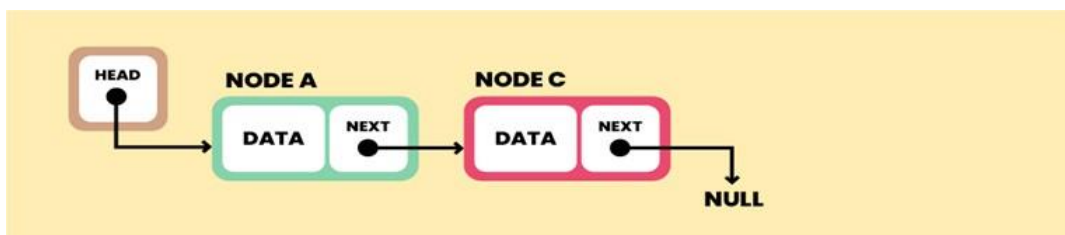


NodeA.next - NodeC

Once, the link pointing to the target node is removed, we'll also remove the link from Node B to Node C.

Now, Node B should point to 'Null'.

Node B.next — Null



Now we can either keep it in memory or deallocate memory and eliminate Node B.

To delete a node from the beginning, we'll simply direct the 'head' to the second node in the list.

In case of deleting the last node, the second last element will be directed to point to Null.

Search

We search for a node on a linked list using a loop. Suppose, we have to find Node X on a list.

- First, we'll assign the 'Head' as the 'Current' node.
- Then, we'll use a loop until the 'Current' node is 'Null' since the last element points to 'Null'.
- In each repetition, we'll check if the key of the node is equal to 'Node X'. If the key matches this item, return true, else return false.

```
void search()
{
    struct node *NewNode; int item,i=0,flag; NewNode = start;
    if(NewNode == NULL)
    {
        printf("\nEmpty List\n");
    }
    else
    {
        printf("\nEnter an item which you want to search\n");
        scanf("%d",&item);
        while (NewNode!=NULL)
        {
            if(NewNode->data == item)
            {
                printf("item found at position%d ",i+1);
                flag=0;
            }
            else
            {
```

```

flag=1;
}
i++;
NewNode = NewNode -> next;
}
if(flag==1)
{
printf("Item not found\n");
}
} }

```

Sort

To sort elements in ascending order, we'll use Bubble Sort. This is how-

- Assign 'Head' as the 'Current' node and create another node index for later use. If 'Head' is null, return
- Otherwise, we'll run a loop until it becomes null.
- In each iteration, we'll store the next node of 'Current' in the index.
- We'll then check if the data of the current node is greater than the next node. In case, it is greater, we'll swap 'Current' and 'Index'. We'll follow the same process throughout.

Let's say we have to sort the numbers - 7, 5, 4, and 6 in ascending order;

// Sort the linked list

```

void sortLinkedList(struct Node** head_ref) { struct Node *current = *head_ref, *index =
NULL; int temp;
if (head_ref == NULL) { return;
} else {
while (current != NULL) {
// index points to the node next to current index = current->next;
while (index != NULL) {
if (current->data > index->data) { temp = current->data;
current->data = index->data; index->data = temp;
}
}
}
}
}

```

```

index = index->next;

}

current = current->next;

} }

}

```

Traversal

In this operation, you will display all the nodes in the linked list. When the temp is null, it means you traversed all the nodes, and you reach the end of the linked list and get out from the while loop.

```

struct node * temp = start; printf("\n list empty are-"); while (temp!= NULL)
{
printf("%d ", temp -> data) temp=temp -> next;
}

```

3.3.3 DOUBLY AND CIRCULARLY LINKED LIST

Doubly Linked List:

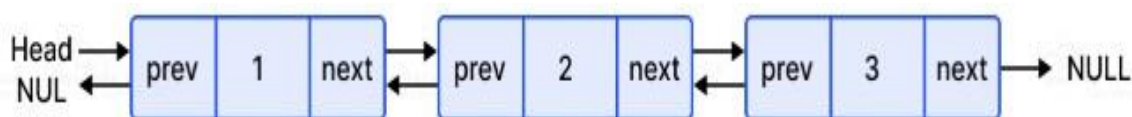
A doubly linked list is a versatile data structure that allows each element, known as a node, to link to both the next and the previous nodes in the sequence. This two-way linking enhances navigation, making it easy to move both forward and backward through the elements.

Unlike singly linked lists, where navigation is only forward, doubly linked lists provide greater flexibility in data management.

For example, in a music player application, a doubly linked list could be used to easily switch between songs, moving forwards or backwards in the playlist.

What is Doubly Linked List in Data Structure?

A doubly linked list is a type of data structure where each element, known as a node, holds data and two references: one pointing to the next node and one pointing to the previous node. This two-way linking allows for movement in both directions through the list, making it more versatile than a singly linked list which only allows movement in one direction.



The importance of doubly linked lists in data structure algorithms (DSA) lies in their ability to facilitate quicker and more flexible modifications—such as insertions and deletions—not just at the ends of the list, but also in the middle.

Example of Doubly Linked List:

The doubly linked list in data structure is particularly beneficial in applications where the ability to navigate backwards is as necessary as moving forwards.

For example, in navigation systems like web browsers, a doubly linked list can manage the user's history, allowing easy forward (redo) and backward (undo) movements through previously visited pages.

This example illustrates how doubly linked lists support complex sequence data management in practical software solutions, enhancing user interface responsiveness and functionality.

Structure of a Doubly Linked List:

Node:

Each node in a doubly linked list includes:

- **Data:** The actual information held within the node, which could be numbers, strings, or any other data type.
- **Next Pointer:** A reference to the next node in the list, which helps in traversing the list forward.
- **Prev Pointer:** A reference to the previous node in the list, which facilitates backward traversal.

**How It Works?**

- **Head and Tail:** The list has two special nodes, the head and the tail. The head points to the first element of the list, while the tail points to the last element. These pointers enhance operations such as adding or removing elements at the ends of the list.
- **Traversal:** You can start from the head to move forward through the list or from the tail to move backward, making it versatile for various operations that require scanning the list in either direction.
- **Insertions and Deletions:** Adding or removing nodes can be done efficiently at any point in the list. When inserting or deleting a node, pointers from adjacent nodes are updated to maintain the list's integrity without needing to shift elements as in contiguous data structures like array.

Difference between Singly and Doubly Linked List

Feature	Singly Linked List	Doubly Linked List
Direction of Traversal	Only forwards. You can traverse from the head to the end of the list.	Both forwards and backwards. You can traverse from the head to the tail and vice versa.
Memory Usage	Less memory per node (one pointer per node).	More memory per node (two pointers per node).
Insertions/Deletions	Efficient at the beginning; requires traversal from the head for other positions.	Efficient at both the beginning and the end; easier insertions and deletions in the middle without full traversal.
Complexity	Simpler and requires less code to manage compared to doubly linked lists.	More complex due to additional pointers, requiring more management and care in code.
Use Case	Suitable when memory is a concern and only forward traversal is needed.	Preferred when frequent operations require elements to be accessed from both ends.
Operations	Typically slower for operations that involve elements at the end of the list.	Faster for operations involving the end of the list due to the tail pointer.
Head and Tail Management	Only head pointer is used.	Both head and tail pointers are used, providing immediate access to both ends of the list.

Doubly Linked List Operations

Operations on doubly linked lists in data structures are more versatile compared to singly linked lists due to their two-way linking of nodes:

1. Insertion

Inserting nodes into a doubly linked list can be done in several ways, each using the ability to access nodes from both directions:

- **At the Beginning:** Add a new node at the start of the list. Set the new node's next pointer to the current head of the list, and update the prev pointer of the existing head to point back to the new node. Update the head pointer to the new node.

- At the End: Add a new node at the end. Traverse to the last node, set the next pointer of the last node to the new node, and set the prev pointer of the new node to the last node. Update the tail pointer if maintained.
- After a Given Node: To insert a node after a given node, adjust the next pointer of the new node to the next pointer of the given node, update the next pointer of the given node to the new node, and set the prev pointer of the node that follows the new node (if any) to the new node.

2. Deletion

Removing nodes requires adjustment of pointers from both the preceding and succeeding nodes:

- From the Beginning: Remove the head node by updating the head to the second node and setting the prev pointer of the new head to null.
- From the End: Remove the tail node by setting the next pointer of the second last node to null and updating the tail to this second last node.
- A Specific Node: Disconnect the node by adjusting the next pointer of the preceding node to point to the node after the target node, and adjust the prev pointer of the succeeding node likewise.

3. Search

Searching involves traversing through the list either from the head or the tail, depending on proximity and potentially the direction of traversal that may optimize the search:

- Search by Value: Start from the head (or tail) and traverse through the next (or prev) pointers to find the node containing the desired value.
- *Traversal*
- Traversal can be performed in both directions, which is a significant advantage of doubly linked lists:
- Forward Traversal: Start from the head and move through each node using the next pointers.
- Backward Traversal: Start from the tail (if available) and move through each node using the prev pointers.

5. Update

Updating the value of a node can be performed directly once the node is accessed, without any specific need for traversal if the node reference is already known.

Advantages of Doubly Linked Lists

- **Bidirectional Navigation:** Allows traversal in both forward and backward directions, facilitating easier and more flexible data manipulation.
- **Easier Insertion and Deletion:** Nodes can be added or removed from both ends and the middle of the list without needing to traverse the entire list, especially if the tail pointer is maintained.
- **Efficient Operations at Both Ends:** Adding or removing elements at the beginning and the end is efficient because both head and tail pointers provide direct access to the endpoints of the list.
- **Dynamic Size:** The size of the list can increase or decrease dynamically, which is efficient for memory usage since it allocates space only as needed.

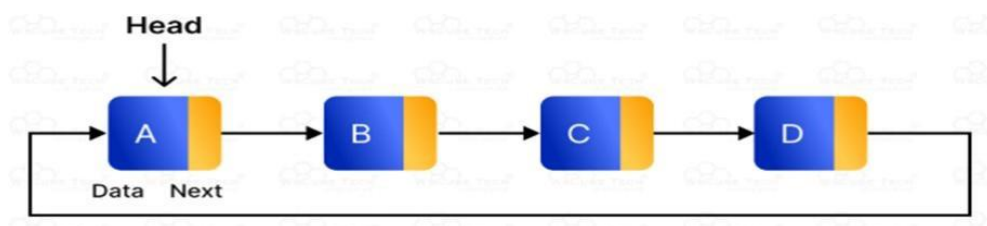
Disadvantages of Doubly Linked Lists

- **Increased Memory Usage:** Each node requires extra memory for an additional pointer (previous pointer), which can be significant in memory-constrained environments.
- **Complexity:** Managing two pointers (next and prev) per node increases the complexity of the operations, making the code more prone to errors such as memory leaks and pointer corruption.
- **Slower Individual Operations:** The overhead of maintaining an extra pointer can slightly slow down operations compared to singly linked lists, as more pointer operations are required.
- **Overhead in Memory Management:** More complex memory management is needed, especially in languages that do not handle garbage collection automatically, due to the additional pointers that need to be correctly handled during insertions and deletions.

Circular Lists

A circular linked list is a type of data structure where each node points to the next node, and the last node points back to the first, forming a circle.

This setup allows for an endless cycle of node traversal, which can be particularly useful in applications where the end of the list naturally reconnects to the beginning.



For example, in computer applications that manage resources in a loop or in creating playlists where the last song leads back to the first song, a circular linked list offers a seamless way to cycle through items without the need for complex logic to manage the end of the list.

Structure of Circular Linked List

A circular linked list in data structure is a variation of the linked list where the last node points back to the first node, making the list circular:

1. Node

Each element in the list is contained in a "node." The structure of a node in a circular linked list includes:

- **Data:** This field holds the actual value that the node represents. This could be any type of data, such as an integer, a string, or even a complex object.
- **Next Pointer:** Unlike in a singly linked list where the last node's next pointer is null, in a circular linked list, the last node's next pointer points back to the first node, creating a loop.

2. Head

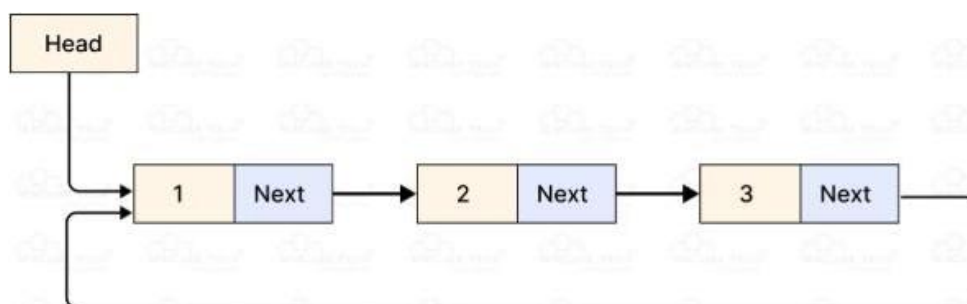
The head is a pointer or reference to the first node in the list. It serves as the entry point for any traversal or modification operation on the list. In a circular linked list, even if you traverse all the way through, you will end up at the head again if you continue to follow each node's next pointer.

Types of Circular Linked List

Circular linked lists can be categorized primarily into two types based on how nodes are linked:

1. Singly Circular Linked List

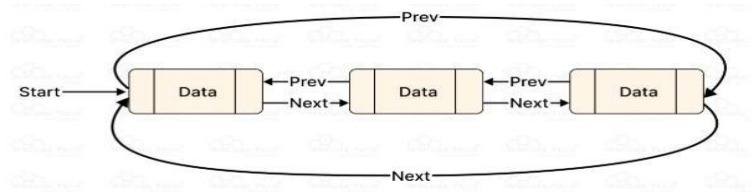
In a singly circular linked list, each node contains a single pointer that points to the next node in the sequence. The unique aspect of this structure is that the last node in the list points back to the first node, forming a circle. This setup allows for continuous, end-to-end traversal of the list.



- **Advantages:** Simple to implement, uses less memory per node (one pointer).
- **Use Cases:** Suitable for applications where the list needs to be traversed in only one direction, such as implementing a round-robin scheduler or a cyclic queue where entries are processed in a continuous loop.

2. Doubly Circular Linked List

Doubly Circular Linked List enhance the singly circular linked list by adding an additional pointer in each node. This second pointer links each node to its previous node, as well as to its next node, forming a two-way link.



Here, both the head and the tail are interconnected, creating a seamless circular navigation system.

- **Advantages:** Allows both forward and backward traversal, making it easier to navigate and modify the list. Useful for more complex data structures where nodes might need to be removed or added frequently and from different positions.
- **Use Cases:** Ideal for applications requiring bidirectional traversal, such as a playlist that allows playing the next or previous media file, or implementing advanced navigation systems in software applications.

Circular Linked List Operations

Here, we will discuss the circular linked operations in data structures:

Search Operation

- The search operation in a circular linked list involves traversing the list from the head and checking each node's data against the search value.
- The traversal continues until the node's data matches the search key or the list loops back to the starting node, indicating that the entire list has been searched.

Delete Operation

- Deleting nodes from a circular linked list can be more complex, especially handling the head and ensuring the circular nature of the list is maintained.

Deleting at the Beginning

To delete a node at the beginning of the list:

- Point a temporary variable to the head.
- Traverse the list to get to the last node (which points to the head).
- Set the last node's next pointer to the head's next node.
- Update the head to be the next node of the current head.
- Free the temporary variable.

Deleting at the End

To delete a node at the end:

- Check if the list is not empty.
- If the list has only one node, remove it and update the head to None.
- Otherwise, traverse to the second-last node.
- Set the second-last node's next pointer to the head.
- Free the last node.

Deleting in the Middle

To delete a node from the middle:

- If the node to delete is the head, follow the steps for deleting at the beginning.
- Otherwise, traverse the list to find the node before the target node.
- Set this node's next pointer to target node's next pointer.
- Free the target node.

Applications of linked list in computer science:

1. Implementation of stacks and queues
2. Implementation of graphs: Adjacency list representation of graphs is the most popular which uses a linked list to store adjacent vertices.
3. Dynamic memory allocation: We use a linked list of free blocks.
4. Maintaining a directory of names
5. Performing arithmetic operations on long integers
6. Manipulation of polynomials by storing constants in the node of the linked list
7. Representing sparse matrices

Applications of linked list in the real world:

1. Image viewer – Previous and next images are linked and can be accessed by the next and previous buttons.
2. Previous and next page in a web browser – We can access the previous and next URL searched in a web browser by pressing the back and next buttons since they are linked as a linked list.
3. Music Player – Songs in the music player are linked to the previous and next songs. So you can play songs either from starting or ending of the list.

4. GPS navigation systems- Linked lists can be used to store and manage a list of locations and routes, allowing users to easily navigate to their desired destination.
5. Robotics- Linked lists can be used to implement control systems for robots, allowing them to navigate and interact with their environment.
6. Task Scheduling- Operating systems use linked lists to manage task scheduling, where each process waiting to be executed is represented as a node in the list.
7. Image Processing- Linked lists can be used to represent images, where each pixel is represented as a node in the list.
8. File Systems- File systems use linked lists to represent the hierarchical structure of directories, where each directory or file is represented as a node in the list.
9. Symbol Table- Compilers use linked lists to build a symbol table, which is a data structure that stores information about identifiers used in a program.
10. Undo/Redo Functionality- Many software applications implement undo/redo functionality using linked lists, where each action that can be undone is represented as a node in a doubly linked list.
11. Speech Recognition- Speech recognition software uses linked lists to represent the possible phonetic pronunciations of a word, where each possible pronunciation is represented as a node in the list.
12. Polynomial Representation- Polynomials can be represented using linked lists, where each term in the polynomial is represented as a node in the list.
13. Simulation of Physical Systems- Linked lists can be used to simulate physical systems, where each element in the list represents a discrete point in time and the state of the system at that time.

Applications of Circular Linked Lists:

1. Useful for implementation of a queue. Unlike this implementation, we don't need to maintain two-pointers for the front and rear if we use a circular linked list. We can maintain a pointer to the last inserted node and the front can always be obtained as next of last.
2. Circular lists are useful in applications to go around the list repeatedly. For example, when multiple applications are running on a PC, it is common for the operating system to put the running applications on a list and then cycle through them, giving each of them a slice of time to execute, and then making them wait while the CPU is given to another application. It is convenient for the operating system to use a circular list so that when it reaches the end of the list it can cycle around to the front of the list.
3. Circular Doubly Linked Lists are used for the implementation of advanced data structures like the Fibonacci Heap.

4. Circular linked lists can be used to implement circular queues, which are often used in operating systems for scheduling processes and managing memory allocation.
5. Used in database systems to implement linked data structures, such as B+ trees, which are used to optimize the storage and retrieval of data.
6. Circular linked lists can be used in networking. For instance, to implement circular buffers for streaming data, such as video and audio, in networking applications.
7. Video games use circular linked lists to manage sprite animations. Each frame of the animation is represented as a node in the list, and the last frame is connected to the first frame to create a loop.
8. Circular linked lists can be used to represent a buffer of audio or signal data in signal processing applications. The last node is connected to the first node to create a loop, and the processing algorithms can efficiently iterate over the data.
9. Traffic light control systems use circular linked lists to manage the traffic light cycles. Each phase of the traffic light cycle is represented as a node in the list, and the last node is connected to the first node to create a loop.

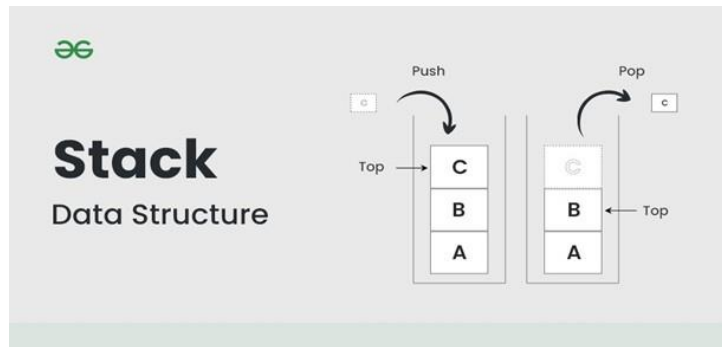
Application of Doubly Linked Lists:

1. Redo and undo functionality.
2. Use of the Back and forward button in a browser.
3. The most recently used section is represented by the Doubly Linked list.
4. Other Data structures like Stack, Hash Table, and Binary Tree can also be applied by Doubly Linked List.
5. Used to implement game objects and their interactions in a game engine.
6. Used in networking.
7. Used in Graph algorithms.
8. Operating systems use doubly linked lists to manage the process scheduling. Each process waiting to be executed is represented as a node in the doubly linked list, and the operating system can easily traverse the list in both directions to manage the process queue.

3.3.4 STACKS

Need for Stack

Stack is a linear data structure that follows a particular order in which the operations are performed. The order may be LIFO (Last In First Out) or FILO(First In Last Out). LIFO implies that the element that is inserted last, comes out first and FILO implies that the element that is inserted first, comes out last.



There are many real-life examples of a stack. Consider an example of plates stacked over one another in the canteen. The plate which is at the top is the first one to be removed, i.e. the plate which has been placed at the bottommost position remains in the stack for the longest period of time. So, it can be simply seen to follow LIFO (Last In First Out)/FILO(First In Last Out) order. Stacks are data structures that allow us to insert and remove items. They operate like a stack of papers or books on our desk - we add new things to the top of the stack to make the stack bigger, and remove items from the top as well to make the stack smaller. This makes stacks a LIFO (Last In First Out) data structure – the data we have put in last is what we will get out first.

The stack data structure is essential in computer science and software engineering for several reasons:

Function Call Management: Stacks are heavily used in managing function calls in programming languages. When a function is called, its local variables, parameters, and return address are pushed onto the stack. When the function execution completes, these elements are popped off the stack. This mechanism allows for nested function calls and supports function return values.

Expression Evaluation: Stacks are used in evaluating arithmetic expressions, converting infix expressions to postfix (or prefix) form, and in evaluating postfix expressions. This is accomplished by using stacks to keep track of operators and operands, ensuring correct precedence and associativity.

Backtracking Algorithms: Many backtracking algorithms, such as depth-first search (DFS), use stacks to keep track of the current state and backtrack when necessary. In DFS, for example, the stack stores the path taken so far, allowing the algorithm to explore deeper into the search space before backtracking.

Parsing and Syntax Analysis: Stacks are used in parsing algorithms such as recursive descent parsers and operator-precedence parsers. These parsers use stacks to keep track of the symbols encountered during parsing and to enforce grammar rules and precedence levels.

Undo Functionality: Stacks are often used to implement undo functionality in text editors, graphics software, and other applications. Each action that modifies the state of the system (e.g., typing, moving objects) is recorded as a command on the stack. Undoing an action involves popping the most recent command from the stack and reverting its effects.

Memory Management: Stacks are used for memory management in computer systems. For example, the call stack in a process stores local variables, function parameters, return addresses, and other data related to function calls. Stacks are also used for managing activation records in operating systems and for implementing stack-based memory allocation strategies.

Expression Parsing and Evaluation: Stacks are used in parsing and evaluating mathematical expressions. By using stacks, expressions can be converted from infix notation to postfix notation, which is easier to evaluate. Stacks can also be used to evaluate postfix expressions efficiently.

Browser History: Stacks are utilized in maintaining browser history. Each URL visited is pushed onto a stack, allowing users to navigate backward (pop) and forward (push) through their browsing history.

Basic features of Stack

- Stack is an ordered list of similar data type.
- Stack is a LIFO (Last in First out) structure or we can say FILO (First in Last out).
- push() function is used to insert new elements into the Stack and pop() function is used to remove an element from the stack. Both insertion and removal are allowed at only one end of Stack called Top.
- Stack is said to be in Overflow state when it is completely full and is said to be in Underflow state if it is completely empty.
- To implement the stack, it is required to maintain the pointer to the top of the stack, which is the last element to be inserted because we can access the elements only on the top of the stack.

Basic operations of Stack:

- **push()** to insert an element into the stack
- **pop()** to remove an element from the stack
- **top()** Returns the top element of the stack.
- **isEmpty()** returns true if stack is empty else false.
- **size()** returns the size of stack.

Push:

Adds an item to the stack. If the stack is full, then it is said to be an Overflow condition.

Algorithm for push:

begin

if stack is full return

```

endif else

increment top stack[top] assign value end else

end procedure

```

Pop:

Removes an item from the stack. The items are popped in the reversed order in which they are pushed. If the stack is empty, then it is said to be an Underflow condition.

Algorithm for pop:

```

begin

if stack is empty return

endif else

store value of stack[top] decrement top

return value end else

end procedure

```

Top:

Returns the top element of the stack.

Algorithm for Top:

```

begin

return stack[top] end procedure isEmpty:

Returns true if the stack is empty, else false.

```

Algorithm for isEmpty:

```

begin

if top < 1 return true

else

return false end procedure

```

Understanding stack practically:

There are many real-life examples of a stack. Consider the simple example of plates stacked over one another in a canteen. The plate which is at the top is the first one to be removed, i.e. the plate which has been placed at the bottommost position remains in the stack for the longest period of time. So, it can be simply seen to follow the LIFO/FILO order.

Complexity Analysis:Time Complexity **Operations** **Complexity push()** **O(1)****pop()** **O(1)****isEmpty()** **O(1)****size()** **O(1)****Types of Stacks:**

Fixed Size Stack: As the name suggests, a fixed size stack has a fixed size and cannot grow or shrink dynamically. If the stack is full and an attempt is made to add an element to it, an overflow error occurs. If the stack is empty and an attempt is made to remove an element from it, an underflow error occurs.

Dynamic Size Stack: A dynamic size stack can grow or shrink dynamically. When the stack is full, it automatically increases its size to accommodate the new element, and when the stack is empty, it decreases its size. This type of stack is implemented using a linked list, as it allows for easy resizing of the stack.

In addition to these two main types, there are several other variations of Stacks, including:

Infix to Postfix Stack: This type of stack is used to convert infix expressions to postfix expressions.

Expression Evaluation Stack: This type of stack is used to evaluate postfix expressions.

Recursion Stack: This type of stack is used to keep track of function calls in a computer program and to return control to the correct function when a function returns.

Memory Management Stack: This type of stack is used to store the values of the program counter and the values of the registers in a computer program, allowing the program to return to the previous state when a function returns.

Balanced Parenthesis Stack: This type of stack is used to check the balance of parentheses in an expression.

Undo-Redo Stack: This type of stack is used in computer programs to allow users to undo and redo actions.

Implementation of Stack:

The basic operations that can be performed on a stack include push, pop, and peek. There are two ways to implement a stack –

- Using array
- Using linked list

In an array-based implementation, the push operation is implemented by incrementing

the index of the top element and storing the new element at that index. The pop operation is implemented by decrementing the index of the top element and returning the value stored at that index.

In a linked list-based implementation, the push operation is implemented by creating a new node with the new element and setting the next pointer of the current top node to the new node. The pop operation is implemented by setting the next pointer of the current top node to the next node and returning the value of the current top node.

Implementing Stack using Arrays:

```

class Stack { int top;
public:
int a[MAX]; // Maximum size of Stack
Stack() { top = -1; } bool push(int x); int pop();
int peek();
bool isEmpty();
};

bool Stack::push(int x)
{
if (top >= (MAX - 1)) {
cout << "Stack Overflow"; return false;
}
else {
a[++top] = x;
cout << x << " pushed into stack\n"; return true;
}
}

int Stack::pop()
{
if (top < 0) {
cout << "Stack Underflow"; return 0;
}
else {

```

```

int x = a[top--]; return x;
}
}

int Stack::peek()
{
if (top < 0) {
cout << "Stack is Empty"; return 0;
}
else {
int x = a[top]; return x;
}
}

bool Stack::isEmpty()
{
return (top < 0);
}

// Driver program to test above functions int main()
{
class Stack s; s.push(10);
s.push(20);
s.push(30);
cout << s.pop() << " Popped from stack\n";
//print top element of stack after popping
cout << "Top element is : " << s.peek() << endl;
//print all elements in stack :
cout << "Elements present in stack : "; while(!s.isEmpty())
{
// print top element in stack cout << s.peek() << " ";
// remove top element in stack s.pop();

```

```

}
return 0;
}

```

Applications of the Stack:

- Infix to Postfix /Prefix conversion
- Redo-undo features at many places like editors, photoshop.
- Forward and backward features in web browsers
- Used in many algorithms like Tower of Hanoi, tree traversals, stock span problems, and histogram problems.
- Backtracking is used to solve problems like the Knight-Tour problem, N-Queen problem, maze problems, and game-like chess or checkers in all these problems we dive into some way if that way is inefficient, we come back to the previous state and go into some another path. To get back from a current state we need to store the previous state in a stack.
- In Graph Algorithms like Topological Sorting and Strongly Connected Components
- In Memory management, any modern computer uses a stack as the primary management for a running purpose. Each program that is running in a computer system has its own memory allocations.
- Stack also helps in implementing function call in computers. The last called function is always completed first.

3.3.5 QUEUES

Queue in data structures is a linear collection of different data types which follow a specific order while performing various operations. It can only be modified by the addition of data entities at one end or the removal of data entities at another. By convention, the end where insertion is performed is called Rear, and the end at which deletion takes place is known as the Front.

These constraints of queue make it a First-In-First-Out (FIFO) data structure, i.e., the data element inserted first will be accessed first, and the data element inserted last will be accessed last. This is equivalent to the requirement that once an additional data element is added, all previously added elements must be removed before the new element can be removed. That's why more abstractly, a queue in a data structure is considered being a sequential collection.

Queue Representation

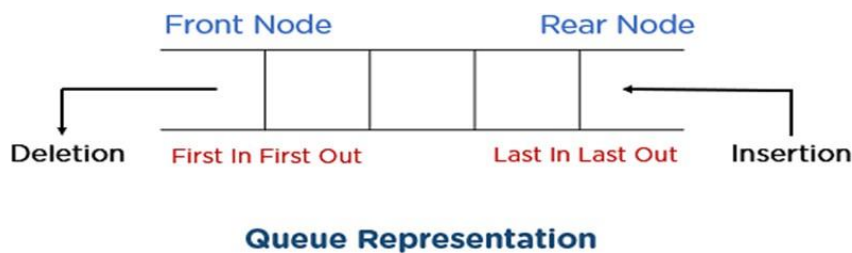
Before dealing with the representation of a queue, examine the real-life example of the queue to understand it better. The movie ticket counter is an excellent example of a queue

where the customer that came first will be served first. Also, the barricades of the movie ticket counter stop in-between disruption to attain different operations at different ends.



The queue in the data structure acts the same as the movie ticket counter. Both the ends of this abstract data structure remain open. Further, the insertion and deletion processes also operate analogously to the wait-up line for tickets.

The following diagram tries to explain queue representation as a data structure:



A queue can be implemented using Arrays, Linked-lists, Pointers, and Structures. The implementation using one-dimensional arrays is the easiest method of all the mentioned methods.

With this understanding of queue representation, look at the different operations that can be performed on the queues in data structures.

Basic Operations for Queue in Data Structure

Unlike arrays and linked lists, elements in the queue cannot be operated from their respective locations. They can only be operated at two data pointers, front and rear. Also, these operations involve standard procedures like initializing or defining data structure, utilizing it, and then wholly erasing it from memory. Here, you must try to comprehend the operations associated with queues:

- Enqueue() - Insertion of elements to the queue.
- Dequeue() - Removal of elements from the queue.
- Peek() - Acquires the data element available at the front node of the queue without deleting it.

- isFull() - Validates if the queue is full.
- isNull() - Checks if the queue is empty.

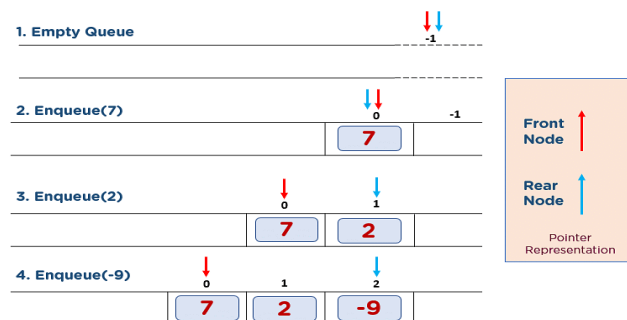
When you define the queue data structure, it remains empty as no element is inserted into it. So, both the front and rear pointer should be set to -1 (Null memory space). This phase is known as data structure declaration in the context of programming.

First, understand the operations that allow the queue to manipulate data elements in a hierarchy.

Enqueue() Operation

The following steps should be followed to insert (enqueue) data element into a queue -

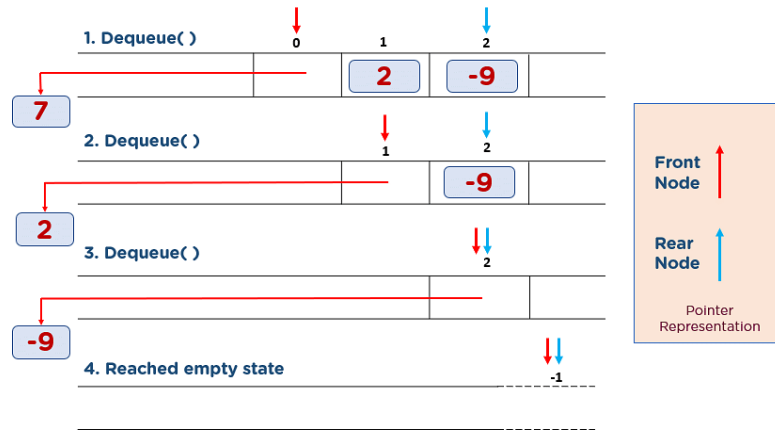
- Step 1: Check if the queue is full.
- Step 2: If the queue is full, Overflow error.
- Step 3: If the queue is not full, increment the rear pointer to point to the next available empty space.
- Step 4: Add the data element to the queue location where the rear is pointing.
- Step 5: Here, you have successfully added 7, 2, and -9.



Dequeue() Operation

Obtaining data from the queue comprises two subtasks: access the data where the front is pointing and remove the data after access. You should take the following steps to remove data from the queue -

- Step 1: Check if the queue is empty.
- Step 2: If the queue is empty, Underflow error.
- Step 3: If the queue is not empty, access the data where the front pointer is pointing.
- Step 4: Increment front pointer to point to the next available data element.
- Step 5: Here, you have removed 7, 2, and -9 from the queue data structure.



Now that you have dealt with the operations that allow manipulation of data entities, you will encounter supportive functions of the queues -

Peek() Operation

This function helps in extracting the data element where the front is pointing without removing it from the queue. The algorithm of Peek() function is as follows-

- Step 1: Check if the queue is empty.
- Step 2: If the queue is empty, return "Queue is Empty."
- Step 3: If the queue is not empty, access the data where the front pointer is pointing.
- Step 4: Return data.

isFull() Operation

This function checks if the rear pointer is reached at MAXSIZE to determine that the queue is full. The following steps are performed in the isFull() operation -

- Step 1: Check if $\text{rear} == \text{MAXSIZE} - 1$.
- Step 2: If they are equal, return "Queue is Full."
- Step 3: If they are not equal, return "Queue is not Full."

isNull() Operation

The algorithm of the isNull() operation is as follows -

- Step 1: Check if the rear and front are pointing to null memory space, i.e., -1.
- Step 2: If they are pointing to -1, return "Queue is empty."
- Step 3: If they are not equal, return "Queue is not empty."

Applications of Queue

Queue, as the name suggests, is utilized when you need to regulate a group of objects in order. This data structure caters to the need for First Come First Serve problems in different

software applications. The scenarios mentioned below are a few systems that use the queue data structure to serve their needs -



- Printers: Queue data structure is used in printers to maintain the order of pages while printing.
- Interrupt handling in computes: The interrupts are operated in the same order as they arrive, i.e., interrupt which comes first, will be dealt with first.
- Process scheduling in Operating systems: Queues are used to implement round-robin scheduling algorithms in computer systems.
- Switches and Routers: Both switch and router interfaces maintain ingress (inbound) and egress (outbound) queues to store packets.
- Customer service systems: It develops call center phone systems using the concepts of queues.

3.4 NON-LINEAR DATA STRUCTURES

A non-linear data structure is a type of data structure where data elements are not stored in a sequence. These structures allow for more complex relationships between data elements, and the elements can have multiple links or branches to other elements. This makes them ideal for representing real-world structures that are hierarchical, or network based.

- A non-linear data structure is another important type in which data elements are not arranged sequentially; mainly, data elements are arranged in random order without forming a linear structure.
- Data elements are present at the multilevel, for example, tree.
- In trees, the data elements are arranged in the hierarchical form, whereas in graphs, the data elements are arranged in random order, using the edges and vertex.

- Multiple runs are required to traverse through all the elements completely. Traversing in a single run is impossible to traverse the whole data structure.
- Each element can have multiple paths to reach another element.
- The data structure where data items are not organized sequentially is called a non-linear data structure. In other words, data elements of the non-linear data structure could be connected to more than one element to reflect a special relationship among them.

Types of Non-Linear Data Structures

There are two types of Non-linear data structures:

1. Tree

A tree is a hierarchical data structure where each element (called a node) is connected to one or more nodes in a parent-child relationship. Trees are often used to represent hierarchical structures, such as file systems or organizational charts.

2. Graph

A graph is a collection of nodes (vertices) and edges, where edges connect the nodes. Graphs are more general and flexible than trees and can represent complex relationships, such as social networks, transportation systems, or web pages.

GRAPH

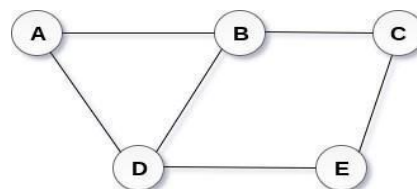
A graph can be defined as group of vertices and edges that are used to connect these vertices. A graph can be seen as a cyclic tree, where the vertices (Nodes) maintain any complex relationship among them instead of having parent child relationship.

Definition

A graph G can be defined as an ordered set $G(V, E)$ where $V(G)$ represents the set of vertices and $E(G)$ represents the set of edges which are used to connect these vertices.

A Graph $G(V, E)$ with 5 vertices (A, B, C, D, E) and six edges ((A,B), (B,C), (C,E), (E,D), (D,B), (D,A)) is shown in the following figure.

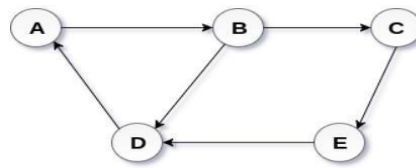
Directed and Undirected Graph



Undirected Graph

A graph can be directed or undirected. However, in an undirected graph, edges are not associated with the directions with them. An undirected graph is shown in the above figure since its edges are not attached with any of the directions. If an edge exists between vertex A and B then the vertices can be traversed from B to A as well as A to B.

In a directed graph, edges form an ordered pair. Edges represent a specific path from some vertex A to another vertex B. Node A is called initial node while node B is called terminal node. A directed graph is shown in the following figure.



Directed Graph

Graph Terminology:

Path

A path can be defined as the sequence of nodes that are followed in order to reach some terminal node V from the initial node U.

Closed Path

A path will be called as closed path if the initial node is same as terminal node. A path will be closed path if $V_0 = V_N$.

Simple Path

If all the nodes of the graph are distinct with an exception $V_0 = V_N$, then such path P is called as closed simple path.

Cycle

A cycle can be defined as the path which has no repeated edges or vertices except the first and last vertices.

Connected Graph

A connected graph is the one in which some path exists between every two vertices (u, v) in V. There are no isolated nodes in connected graph.

Complete Graph

A complete graph is the one in which every node is connected with all other nodes. A complete graph contains $n(n-1)/2$ edges where n is the number of nodes in the graph.

Weighted Graph

In a weighted graph, each edge is assigned with some data such as length or weight. The weight of an edge e can be given as $w(e)$ which must be a positive (+) value indicating the cost of traversing the edge.

Digraph

A digraph is a directed graph in which each edge of the graph is associated with some direction and the traversing can be done only in the specified direction.

Loop

An edge that is associated with the similar end points can be called as Loop.

Adjacent Nodes

If two nodes u and v are connected via an edge e , then the nodes u and v are called as neighbors or adjacent nodes.

Degree of the Node

A degree of a node is the number of edges that are connected with that node. A node with degree 0 is called as isolated node.

Representations of Graph

Here are the two most common ways to represent a graph :

- Adjacency Matrix
- Adjacency List

Adjacency Matrix

An adjacency matrix is a way of representing a graph as a matrix of boolean (0's and 1's). Let's assume there are n vertices in the graph So, create a 2D matrix $\text{adjMat}[n][n]$ having dimension $n \times n$.

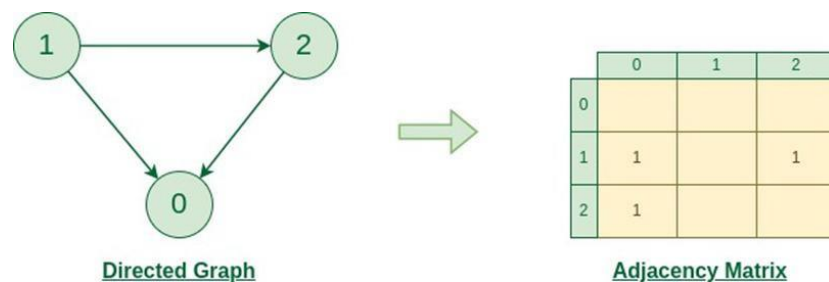
- If there is an edge from vertex i to j , mark $\text{adjMat}[i][j]$ as 1.
- If there is no edge from vertex i to j , mark $\text{adjMat}[i][j]$ as 0.

Representation of Undirected Graph to Adjacency Matrix:

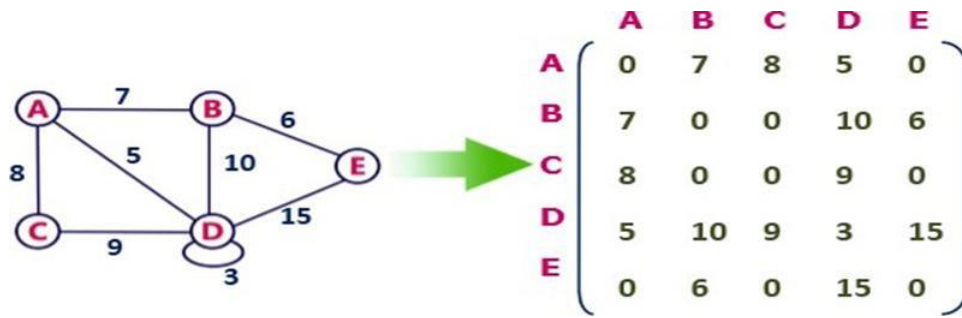
The below figure shows an undirected graph. Initially, the entire Matrix is initialized to 0. If there is an edge from source to destination, we insert 1 to both cases ($\text{adjMat}[\text{destination}]$ and $\text{adjMat}[\text{source}]$) because we can go either way.

Representation of Directed Graph to Adjacency Matrix:

The below figure shows a directed graph. Initially, the entire Matrix is initialized to 0. If there is an edge from source to destination, we insert 1 for that particular $\text{adjMat}[\text{destination}]$.



Weighted Graph Representation



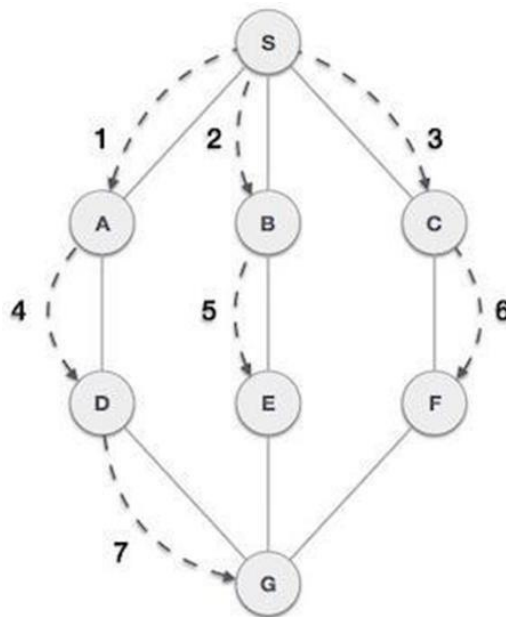
Graph Traversal in Data Structure

We can traverse a graph in two ways:

1. BFS (Breadth First Search)
2. DFS (Depth First Search)

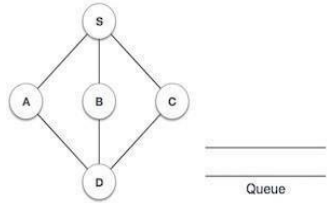
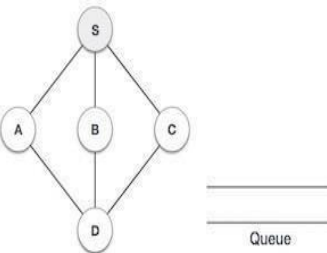
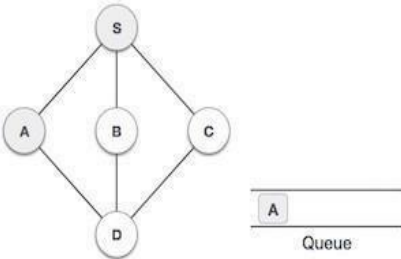
Breadth First Search

Breadth First Search (BFS) algorithm traverses a graph in a breadth ward motion and uses a queue to remember to get the next vertex to start a search, when a dead end occurs in any iteration.



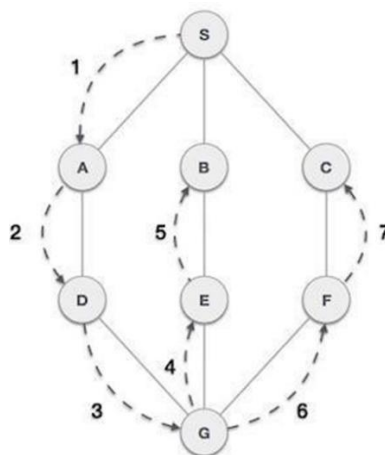
As in the example given above, BFS algorithm traverses from A to B to E to F first then to C and G lastly to D. It employs the following rules.

- **Rule 1** – Visit the adjacent unvisited vertex. Mark it as visited. Display it. Insert it in a queue.
- **Rule 2** – If no adjacent vertex is found, remove the first vertex from the queue.
- **Rule 3** – Repeat Rule 1 and Rule 2 until the queue is empty.

Step	Traversal	Description
1		Initialize the queue.
2		We start from visiting S (starting node), and mark it as visited.
3		We then see an unvisited adjacent node from S . In this example, we have three nodes but alphabetically we choose A , mark it as visited and enqueue it.

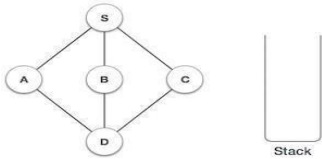
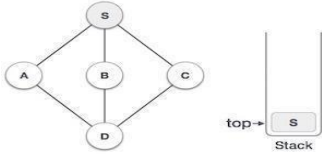
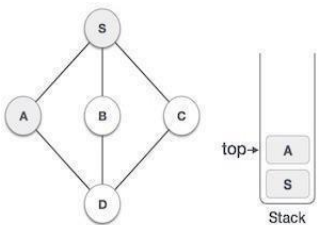
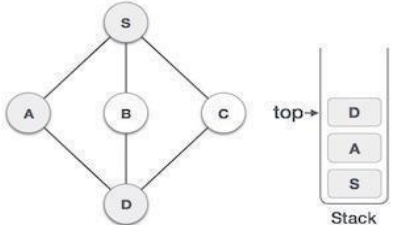
Depth First Search

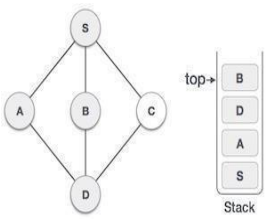
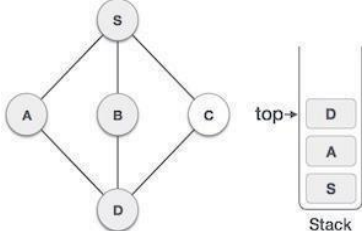
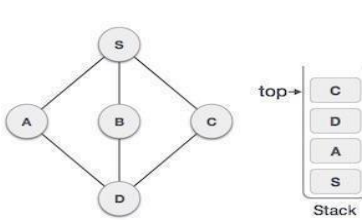
Depth First Search (DFS) algorithm traverses a graph in a depth ward motion and uses a stack to remember to get the next vertex to start a search, when a dead end occurs in any iteration.



As in the example given above, DFS algorithm traverses from S to A to D to G to E to B first, then to F and lastly to C. It employs the following rules.

- **Rule 1** – Visit the adjacent unvisited vertex. Mark it as visited. Display it. Push it in a stack.
- **Rule 2** – If no adjacent vertex is found, pop up a vertex from the stack. (It will pop up all the vertices from the stack, which do not have adjacent vertices.)
- **Rule 3** – Repeat Rule 1 and Rule 2 until the stack is empty

Step	Traversal	Description
1		Initialize the stack.
2		Mark S as visited and put it onto the stack. Explore any unvisited adjacent node from S . We have three nodes and we can pick any of them. For this example, we shall take the node in an alphabetical order.
3		Mark A as visited and put it onto the stack. Explore any unvisited adjacent node from A . Both S and D are adjacent to A but we are concerned for unvisited nodes only.
4		Visit D and mark it as visited and put onto the stack. Here, we have B and C nodes, which are adjacent to D and both are unvisited. However, we shall again choose in an alphabetical order.

5		We choose B, mark it as visited and put onto the stack. Here B does not have any unvisited adjacent node. So, we pop B from the stack.
6		We check the stack top for return to the previous node and check if it has any unvisited nodes. Here, we find D to be on the top of the stack.
7		Only unvisited adjacent node is from D is C now. So we visit C, mark it as visited and put it onto the stack.

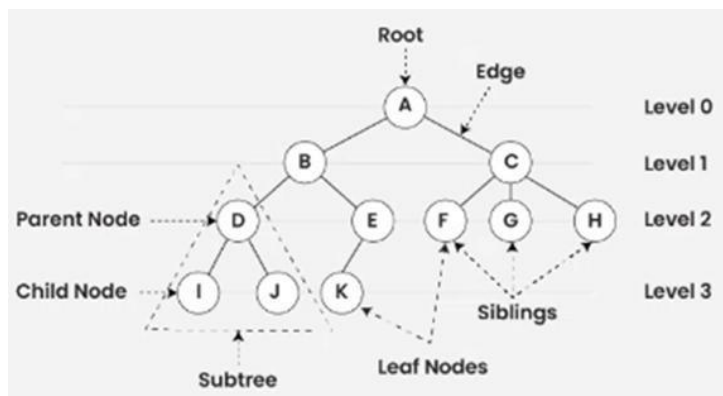
3.4.1 TREES

Introduction to Trees

Tree data structure is a hierarchical structure that is used to represent and organize data in the form of parent child relationship. The following are some real world situations which are naturally a tree.

- Folder structure in an operating system.
- Tag structure in an HTML (root tag the as html tag) or XML document.

The topmost node of the tree is called the **root**, and the nodes below it are called the child nodes. Each node can have multiple child nodes, and these child nodes can also have their own child nodes, forming a recursive structure.

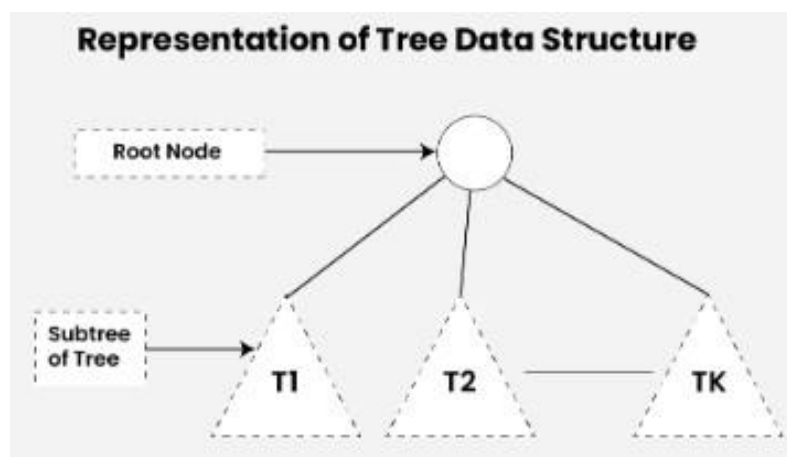


Basic Terminologies in Tree Data Structure:

- **Parent Node:** The node which is an immediate predecessor of a node is called the parent node of that node. **{B}** is the parent node of **{D, E}**.
- **Child Node:** The node which is the immediate successor of a node is called the child node of that node. Examples: **{D, E}** are the child nodes of **{B}**.
- **Root Node:** The topmost node of a tree or the node which does not have any parent node is called the root node. **{A}** is the root node of the tree. A non-empty tree must contain exactly one root node and exactly one path from the root to all other nodes of the tree.
- **Leaf Node or External Node:** The nodes which do not have any child nodes are called leaf nodes. **{I, J, K, F, G, H}** are the leaf nodes of the tree.
- **Ancestor of a Node:** Any predecessor nodes on the path of the root to that node are called Ancestors of that node. **{A,B}** are the ancestor nodes of the node **{E}**
- **Descendant:** A node x is a descendant of another node y if and only if y is an ancestor of x.
- **Sibling:** Children of the same parent node are called siblings. **{D,E}** are called siblings.
- **Level of a node:** The count of edges on the path from the root node to that node. The root node has level 0.
- **Internal node:** A node with at least one child is called Internal Node.
- **Neighbour of a Node:** Parent or child nodes of that node are called neighbors of that node.
- **Subtree:** Any node of the tree along with its descendant.

Representation of Tree Data Structure:

A tree consists of a root node, and zero or more subtrees $T_1, T_2, \dots, T_k$ such that there is an edge from the root node of the tree to the root node of each subtree. Subtree of a node X consists of all the nodes which have node X as the ancestor node.



Types of Tree data structures:

Tree data structure can be classified into three types based upon the number of children each node of the tree can have. The types are:

- **Binary tree:** In a binary tree, each node can have a maximum of two children linked to it. Some common types of binary trees include full binary trees, complete binary trees, balanced binary trees, and degenerate or pathological binary trees. Examples of Binary Tree are Binary Search Tree and Binary Heap.
- **Ternary Tree:** A Ternary Tree is a tree data structure in which each node has at most three child nodes, usually distinguished as “left”, “mid” and “right”.
- **N-ary Tree or Generic Tree:** Generic trees are a collection of nodes where each node is a data structure that consists of records and a list of references to its children (duplicate references are not allowed). Unlike the linked list, each node stores the address of multiple nodes.

Basic Operations of Tree Data Structure:

- **Create** – create a tree in the data structure.
- **Insert** – Inserts data in a tree.
- **Search** – Searches specific data in a tree to check whether it is present or not.
- **Traversal** - traversing or visiting each node of a tree

Tree Traversal

The term 'tree traversal' means traversing or visiting each node of a tree. There is a single way to traverse the linear data structure such as linked list, queue, and stack. Whereas, there are multiple ways to traverse a tree that are listed as follows -

- Preorder traversal
- Inorder traversal
- Postorder traversal

Preorder traversal

This technique follows the 'root left right' policy. It means that, first root node is visited after that the left subtree is traversed recursively, and finally, right subtree is recursively traversed. As the root node is traversed before (or pre) the left and right subtree, it is called preorder traversal. So, in a preorder traversal, each node is visited before both of its subtrees.

The applications of preorder traversal include -

- It is used to create a copy of the tree.
- It can also be used to get the prefix expression of an expression tree.

Algorithm

Until all nodes of the tree are not visited

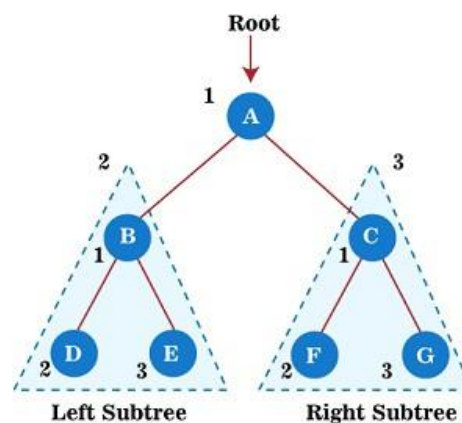
Step 1 - Visit the root node

Step 2 - Traverse the left subtree recursively.

Step 3 - Traverse the right subtree recursively.

Example

Now, let's see the example of the preorder traversal technique.



Now, start applying the preorder traversal on the above tree. First, we traverse the root node **A**; after that, move to its left subtree **B**, which will also be traversed in preorder.

So, for left subtree B, first, the root node **B** is traversed itself; after that, its left subtree **D** is traversed. Since node **D** does not have any children, move to right subtree **E**. As node E also does not have any children, the traversal of the left subtree of root node A is completed.

Now, move towards the right subtree of root node A that is C. So, for right subtree C, first the root node **C** has traversed itself; after that, its left subtree **F** is traversed. Since node **F** does not have any children, move to the right subtree **G**. As node G also does not have any children, traversal of the right subtree of root node A is completed.

Therefore, all the nodes of the tree are traversed. So, the output of the preorder traversal of the above tree is -

A → B → D → E → C → F → G

Postorder traversal

This technique follows the 'left-right root' policy. It means that the first left subtree of the root node is traversed, after that recursively traverses the right subtree, and finally, the root node is traversed.

As the root node is traversed after (or post) the left and right subtree, it is called postorder traversal. So, in a postorder traversal, each node is visited after both of its subtrees.

The applications of postorder traversal include -

- It is used to delete the tree.
- It can also be used to get the postfix expression of an expression tree.

Algorithm

Until all nodes of the tree are not visited

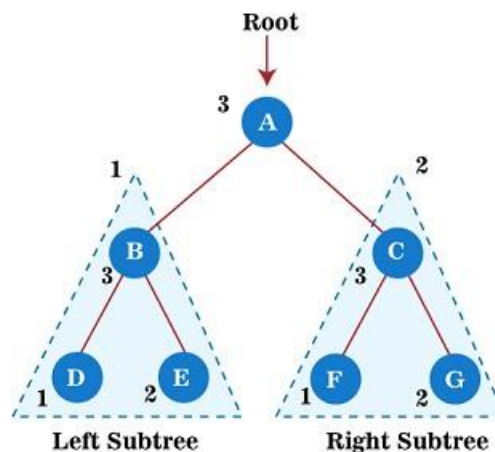
Step 1 - Traverse the left subtree recursively.

Step 2 - Traverse the right subtree recursively.

Step 3 - Visit the root node.

Example

Now, let's see the example of the postorder traversal technique.



Now, start applying the postorder traversal on the above tree. First, we traverse the left subtree B that will be traversed in postorder. After that, we will traverse the right subtree C in postorder. And finally, the root node of the above tree, i.e., A, is traversed.

So, for left subtree B, first, its left subtree D is traversed. Since node D does not have any children, traverse the right subtree E. As node E also does not have any children, move to the root node B. After traversing node B, the traversal of the left subtree of root node A is completed.

Now, move towards the right subtree of root node A that is C. So, for right subtree C, first its left subtree F is traversed. Since node F does not have any children, traverse the right subtree G. As node G also does not have any children, therefore, finally, the root node of the right subtree, i.e., C, is traversed. The traversal of the right subtree of root node A is completed.

At last, traverse the root node of a given tree, i.e., A. After traversing the root node, the postorder traversal of the given tree is completed. Therefore, all the nodes of the tree are traversed. So, the output of the postorder traversal of the above tree is -

D → E → B → F → G → C → A

Inorder traversal

This technique follows the 'left root right' policy. It means that first left subtree is visited after that root node is traversed, and finally, the right subtree is traversed. As the root node is traversed between the left and right subtree, it is named inorder traversal.

So, in the inorder traversal, each node is visited in between of its subtrees.

The applications of Inorder traversal includes -

- It is used to get the BST nodes in increasing order.
- It can also be used to get the prefix expression of an expression tree.

Algorithm

Until all nodes of the tree are not visited

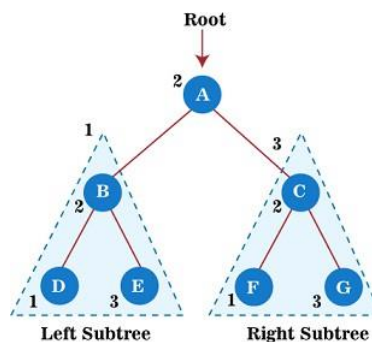
Step 1 - Traverse the left subtree recursively.

Step 2 - Visit the root node.

Step 3 - Traverse the right subtree recursively.

Example

Now, let's see the example of the Inorder traversal technique.



Now, start applying the inorder traversal on the above tree. First, we traverse the left subtree **B** that will be traversed in inorder. After that, we will traverse the root node **A**. And finally, the right subtree **C** is traversed in inorder.

So, for left subtree **B**, first, its left subtree **D** is traversed. Since node **D** does not have any children, so after traversing it, node **B** will be traversed, and at last, right subtree of node B, that is **E**, is traversed. Node E also does not have any children; therefore, the traversal of the left subtree of root node A is completed.

After that, traverse the root node of a given tree, i.e., **A**.

At last, move towards the right subtree of root node A that is C. So, for right subtree C; first, its left subtree **F** is traversed. Since node **F** does not have any children, node **C** will be traversed, and at last, a right subtree of node C, that is, **G**, is traversed. Node G also does not have any children; therefore, the traversal of the right subtree of root node A is completed.

As all the nodes of the tree are traversed, the inorder traversal of the given tree is completed. The output of the inorder traversal of the above tree is -

D → B → E → A → F → C → G

3.4.2 BINARY TREES

A binary tree in DSA (Data Structures and Algorithms) is a way to organize data in a hierarchical structure. In a binary tree, each node has at most two children, called the left child and the right child. The topmost node is called the root, and the nodes with no children are called leaves.

The basic idea of a binary tree is to have a parent-child relationship between nodes. Each node can have a left and a right child, and this pattern continues down the tree. This structure makes it easy to organize and find data quickly.

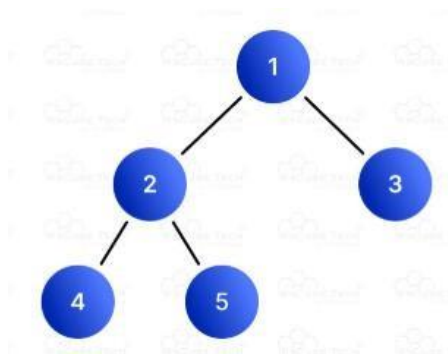
Types of Binary Tree in Data Structure

Let's understand what are the different types of binary tree with examples:

Full Binary Tree

A full binary tree is a tree where every node has either 0 or 2 children.

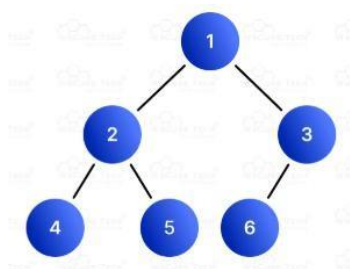
Example:



Complete Binary Tree

A complete binary tree is a tree where all levels are fully filled except possibly the last level, which is filled from left to right.

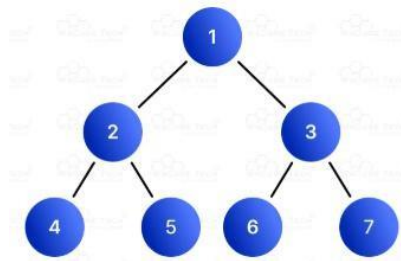
Example:



Perfect Binary Tree

A perfect binary tree is a tree where all internal nodes have exactly two children and all leaf nodes are at the same level.

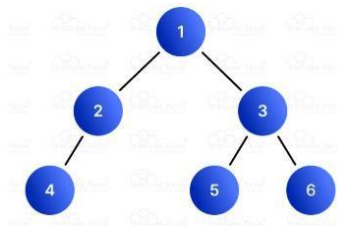
Example:



Balanced Binary Tree

A balanced binary tree is a tree where the height of the left and right subtrees of any node differ by at most one.

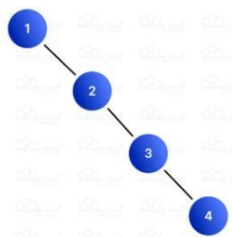
Example:



Degenerate (or Pathological) Binary Tree

A degenerate binary tree is a tree where each parent node has only one child. This makes the tree look like a linked list.

Example:



Binary Tree Operations with Examples

Binary trees support several fundamental operations, including insertion, deletion, searching, and traversal:

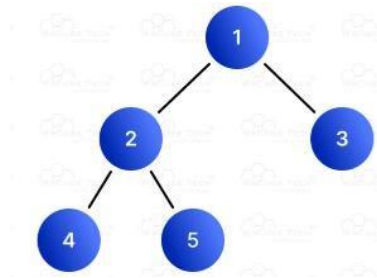
Insertion

Insertion involves adding a new node to the binary tree. In a binary tree, a new node is

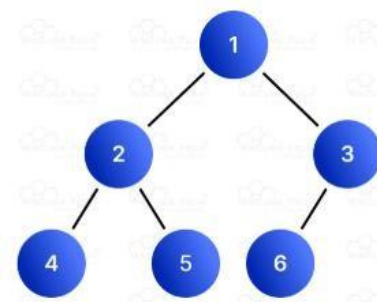
usually inserted at the first available position in level order to maintain the completeness of the tree.

Example:

Let's insert the value 6 into the following binary tree:



After insertion, the binary tree will look like this:

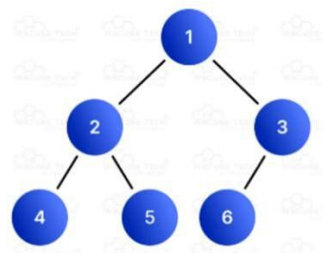


Deletion

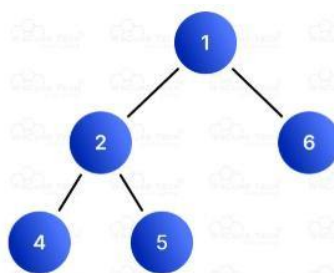
Deletion involves removing a node from the binary tree. In a binary tree, the node to be deleted is replaced by the deepest and rightmost node to maintain the tree's structure.

Example:

Let's delete the value 3 from the following binary tree:



After deletion, the binary tree will look like this:

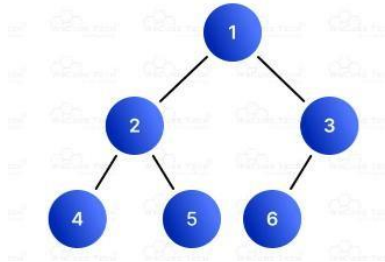


Search

Searching involves finding a node with a given value in the binary tree. The search operation can be implemented using any traversal method (in-order, pre-order, post-order, or level-order).

Example:

Let's search for the value 5 in the following binary tree:



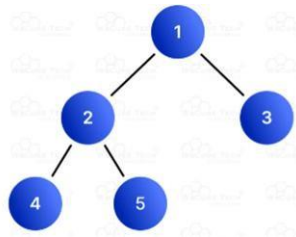
Using level-order traversal:

- Visit node 1
- Visit node 2
- Visit node 3
- Visit node 4
- Visit node 5 (found)

Traversal

Traversal involves visiting all the nodes in the binary tree in a specific order. The main traversal methods are in-order, pre-order, post-order, and level-order.

Example:



Consider the above binary tree:

- In-order Traversal (Left, Root, Right): 4, 2, 5, 1, 3
- Pre-order Traversal (Root, Left, Right): 1, 2, 4, 5, 3
- Post-order Traversal (Left, Right, Root): 4, 5, 2, 3, 1
- Level-order Traversal (Breadth-First): 1, 2, 3, 4, 5

Time and Space Complexity of Binary Tree

Operation	Worst Case Time Complexity	Space Complexity
Insertion	$O(n)$	$O(n)$
Deletion	$O(n)$	$O(n)$
Search	$O(n)$	$O(n)$
Traversal (In-order, Pre-order, Post-order)	$O(n)$	$O(h)$ (where h is the height of the tree)

Applications of Binary Trees with Examples

Binary trees have a wide range of applications in computer science and various algorithms:

- Binary Search Trees (BST): Efficient searching, insertion, and deletion.
- Heaps: Implement priority queues with efficient maximum or minimum retrieval.
- Expression Trees: Represent and evaluate arithmetic expressions.
- Huffman Coding Tree: Data compression using optimal prefix codes.
- AVL Trees: Self-balancing BSTs for efficient operations.
- B-Trees: Used in databases and file systems for balanced storage.
- Decision Trees: Machine learning models for classification and regression.